```
// ==================== 文件: src/App.tsx ====================
import { lazy, Suspense, useEffect, useLayoutEffect, createContext, useContext, useRef, useState, type ReactNode } from 'react'
import { HashRouter, Routes, Route, Link, useNavigate, useLocation } from 'react-router-dom'
import { AuthProvider, useAuth } from './contexts/AuthContext'
import { UpdateProvider } from './contexts/UpdateContext'
import { LogsProvider } from './contexts/LogsContext'
import Navbar from './components/Navbar'
import BottomTab from './components/BottomTab'
import ProtectedRoute from './components/ProtectedRoute'
import UpdateChecker from './components/UpdateChecker'
import AchievementNotifier from './components/AchievementNotifier'
import CheckinReminder from './components/CheckinReminder'
import { ToastProvider } from './lib/Toast'
import { Capacitor } from '@capacitor/core'
import { App as CapacitorApp } from '@capacitor/app'

// 路由级代码分割：按需加载页面，减少首屏 bundle 体积
const Home = lazy(() => import('./pages/Home'))
const Login = lazy(() => import('./pages/Login'))
const Register = lazy(() => import('./pages/Register'))
const ForgotPassword = lazy(() => import('./pages/ForgotPassword'))
const ResetPassword = lazy(() => import('./pages/ResetPassword'))
const MyRecords = lazy(() => import('./pages/MyRecords'))
const NewRecord = lazy(() => import('./pages/NewRecord'))
const EditRecord = lazy(() => import('./pages/EditRecord'))
const Summary = lazy(() => import('./pages/Summary'))
const TimerPage = lazy(() => import('./pages/TimerPage'))
const Profile = lazy(() => import('./pages/Profile'))
const Trash = lazy(() => import('./pages/Trash'))
const Achievements = lazy(() => import('./pages/Achievements'))
const GoalPage = lazy(() => import('./pages/GoalPage'))
const EnglishCheckin = lazy(() => import('./pages/EnglishCheckin'))
const VocabularyBook = lazy(() => import('./pages/VocabularyBook'))
const Settings = lazy(() => import('./pages/Settings'))
const Health = lazy(() => import('./pages/Health'))
const QrLogin = lazy(() => import('./pages/QrLogin'))
const ScanQr = lazy(() => import('./pages/ScanQr'))

/** 首页布局上下文：桌面端「全部功能」全功能模式下强制双栏布局 */
export const HomeLayoutContext = createContext<{ twoCol: boolean; wide: boolean }>({ twoCol: false, wide: false })

/** 页面内容是否放宽到更宽的布局（桌面端全功能模式下） */
export function useWideLayout(): boolean {
  return useContext(HomeLayoutContext).wide
}

/** 自适应缩放的下限兜底：极端窄高窗口下也不至于缩到看不清 */
const MIN_FIT_SCALE = 0.4

/**
 * 桌面「全部功能」模式下，让页面内容按内容高度自适应缩放填满窗口。
 * 首页本身已用 forceTwoCol 双栏单屏，无需缩放，故跳过首页。
 *
```

```
 * 当内容高度超过视口 1.2 倍时，允许纵向滚动而非继续缩小，保证长页面可读性。
 *
 * 关键约束：内容层必须始终保持固定布局宽度（100%），绝不能用 `width: 100% / scale`
 * 反向补偿缩放后的视觉宽度。否则会形成正反馈——
 * scale 变小 → 布局变宽 → aspect-square / aspect-[5/4] 这类元素高度变大 → scrollHeight 变大
 * → scale 更小；再加上宽度兜底分支 `ow / scrollWidth` 恒 ≤ 当前 scale（因为
 * scrollWidth ≥ clientWidth = ow / scale），scale 就成了只能降不能升的单向棘轮，
 * 页面会一路缩小下去。固定布局宽度后，测量值与已应用的缩放无关，结果天然收敛。
 */
function ScaleToFit({ enabled, children }: { enabled: boolean; children: ReactNode }) {
  const location = useLocation()
  const outerRef = useRef<HTMLDivElement>(null)
  const innerRef = useRef<HTMLDivElement>(null)
  const [scale, setScale] = useState(1)

  useLayoutEffect(() => {
    if (!enabled) {
      setScale(1)
      return
    }
    const outer = outerRef.current
    const inner = innerRef.current
    if (!outer || !inner) return

    let raf = 0
    // rAF 节流：合并同一帧内的多次回调，避免 ResizeObserver 抖动与循环告警
    const apply = () => {
      cancelAnimationFrame(raf)
      raf = requestAnimationFrame(() => {
        const ow = outer.clientWidth
        const oh = outer.clientHeight
        const ih = inner.scrollHeight
        const iw = inner.scrollWidth
        if (ow <= 0 || oh <= 0 || ih <= 0 || iw <= 0) return
        // 内容超高（超过视口 1.2 倍）时允许滚动，不再继续缩小
        if (ih > oh * 1.2) {
          setScale(1)
          return
        }
        // 按高度缩放，同时用宽度兜底保证不横向溢出；不放大（上限 1）
        const next = Math.max(MIN_FIT_SCALE, Math.min(1, oh / ih, ow / iw))
        // 变化极小则忽略，避免浮点误差引起无意义的重渲染
        setScale((prev) => (Math.abs(prev - next) < 0.005 ? prev : next))
      })
    }

    apply()
    const ro = new ResizeObserver(apply)
    ro.observe(outer)
    ro.observe(inner)
    return () => {
      cancelAnimationFrame(raf)
      ro.disconnect()
    }
```

```
  }, [location.pathname, enabled])

  // 非桌面全功能模式，或首页已双栏单屏，直接渲染不缩放
  if (!enabled || location.pathname === '/') return <>{children}</>

  return (
    <div ref={outerRef} className="w-full h-full overflow-y-auto min-h-0">
      <div
        ref={innerRef}
        style={{ transform: `scale(${scale})`, transformOrigin: 'top center', width: '100%' }}
      >
        {children}
      </div>
    </div>
  )
}

function PageLoading() {
  return (
    <div className="flex min-h-[40vh] items-center justify-center">
      <div className="h-8 w-8 animate-spin rounded-full border-4 border-blue-200 dark:border-slate-700 border-t-blue-600 dark:border-t-blue-500" />
    </div>
  )
}

function NotFound() {
  return (
    <div className="mx-auto max-w-2xl px-4 py-16 text-center">
      <p className="text-6xl font-bold text-gray-200 dark:text-slate-700 mb-4">404</p>
      <p className="text-lg text-gray-600 dark:text-slate-300 mb-6">页面不存在或已被移除</p>
      <Link
        to="/"
        className="inline-block px-5 py-2.5 bg-blue-600 hover:bg-blue-700 text-white rounded-lg text-sm font-medium transition-colors cursor-pointer"
      >
        返回首页
      </Link>
    </div>
  )
}

/** 全局配置/认证错误横幅（如 Supabase 未配置时给出明确提示，避免无感知白屏） */
function ConfigBanner() {
  const { error } = useAuth()
  if (!error) return null
  return (
    <div className="bg-amber-100 dark:bg-amber-900/40 border-b border-amber-200 dark:border-amber-700/50 text-amber-800 dark:text-amber-200 text-center text-sm py-2 px-4">
      {error}
    </div>
  )
}

/** Capacitor 原生返回按钮处理：在 App 内导航回退，而非直接退出应用 */
```

```
160 function BackButtonHandler() {
161   const navigate = useNavigate()
162   useEffect(() => {
163     if (!Capacitor.isNativePlatform()) return
164     const handler = CapacitorApp.addListener('backButton', () => {
165       // 如果当前页面不是首页，则导航回退；否则最小化应用
166       if (window.location.hash !== '#/') {
167         navigate(-1)
168       } else {
169         CapacitorApp.minimizeApp()
170       }
171     })
172     return () => {
173       handler.then((h) => h.remove())
174     }
175   }, [navigate])
176   return null
177 }
178
179 export default function App({
180   hideBottomTab = false,
181   hideNavbar = false,
182   sidebar = null,
183   fillHeight = false,
184   forceTwoCol = false,
185 }: {
186   hideBottomTab?: boolean
187   hideNavbar?: boolean
188   /** 需要侧边栏导航时传入（如桌面端展开模式）；必须在 Router 内部渲染，故由 App 接收）
189    */ sidebar?: ReactNode | null
190   /** 桌面展开模式下用 100% 高度替代 100vh，配合滚动容器避免页面底部被裁切 */
191   fillHeight?: boolean
192   /** 桌面「全部功能」全功能模式下强制首页双栏布局（无视视口宽度） */
193   forceTwoCol?: boolean
194 }) {
195   // 桌面展开模式（fillHeight）：用确定高度 h-full 替代 min-h，让百分比高度链条成立，
196   // 侧边栏才能撑满整列高度并把底部按钮钉在可见区最底部；移动端保持 min-h-screen 原逻辑
197   const pageHeight = fillHeight ? 'h-full' : 'min-h-screen'
198   return (
199     <HashRouter>
200       <ToastProvider>
201         <BackButtonHandler />
202         <UpdateProvider>
203           <AuthProvider>
204           <LogsProvider>
205           <div className={`relative isolate overflow-x-clip ${pageHeight} theme-surface bg-gra
    y-50 text-gray-900 dark:bg-slate-950 dark:text-slate-100 transition-colors duration-200 pb-16 sm:pb-
    0`}>
206           {/* 顶部环境光：页面顶部分层淡出，增加呼吸感 */}
207           <div
208             aria-hidden
209             className="pointer-events-none absolute inset-x-0 top-0 h-80 bg-gradient-to-b from
    -indigo-100/70 via-indigo-50/40 to-transparent dark:from-indigo-500/10 dark:via-indigo-900/5 dark:to
    -transparent"
210           />
```

```
211           <div className={`relative flex ${pageHeight}`}>
212             {/* 侧边栏：stretch 撑满整列高度，配合 sticky 让底部的主题/登出始终固定在可见区最
    底部 */}
213             {sidebar && <div className="sticky top-0 shrink-0 self-stretch">{sidebar}</div>}
214             <div className="flex-1 min-w-0">
215               <ConfigBanner />
216               <UpdateChecker />
217               <AchievementNotifier />
218               <CheckinReminder />
219               {!hideNavbar && <Navbar />}
220               <Suspense fallback={<PageLoading />}>
221                 <HomeLayoutContext.Provider value={{ twoCol: forceTwoCol, wide: forceTwoCol }}
    >
222                 <ScaleToFit enabled={forceTwoCol}>
223                 <Routes>
224                   <Route path="/" element={<Home />} />
225                   <Route path="/login" element={<Login />} />
226                   <Route path="/register" element={<Register />} />
227                   <Route path="/forgot-password" element={<ForgotPassword />} />
228                   <Route path="/reset-password" element={<ResetPassword />} />
229                   <Route path="/qr-login" element={<QrLogin />} />
230                   <Route element={<ProtectedRoute />}>
231                     <Route path="/my-records" element={<MyRecords />} />
232                     <Route path="/profile" element={<Profile />} />
233                     <Route path="/summary" element={<Summary />} />
234                     <Route path="/my-records/new" element={<NewRecord />} />
235                     <Route path="/my-records/:id/edit" element={<EditRecord />} />
236                     <Route path="/timer" element={<TimerPage />} />
237                     <Route path="/trash" element={<Trash />} />
238                     <Route path="/achievements" element={<Achievements />} />
239                     <Route path="/goal" element={<GoalPage />} />
240                     <Route path="/english-checkin" element={<EnglishCheckin />} />
241                     <Route path="/vocabulary" element={<VocabularyBook />} />
242                     <Route path="/settings" element={<Settings />} />
243                     <Route path="/health" element={<Health />} />
244                     <Route path="/scan-qr" element={<ScanQr />} />
245                   </Route>
246                   <Route path="*" element={<NotFound />} />
247                 </Routes>
248                 </ScaleToFit>
249                 </HomeLayoutContext.Provider>
250               </Suspense>
251               {!hideBottomTab && <BottomTab />}
252             </div>
253           </div>
254           </div>
255           </LogsProvider>
256       </AuthProvider>
257       </UpdateProvider>
258       </ToastProvider>
259     </HashRouter>
260   )
261 }
262
263 // ==================== 文件: src/main.tsx ====================
```

```
264 import { StrictMode, useEffect, useState } from 'react'
265 import { createRoot } from 'react-dom/client'
266 import './index.css'
267 import { isTauri } from '@tauri-apps/api/core'
268 import App from './App.tsx'
269 import WidgetApp from './widget/WidgetApp'
270 import Sidebar from './components/Sidebar'
271 import DesktopLogo from './components/DesktopLogo'
272 import { AuthProvider } from './contexts/AuthContext'
273 import { ToastProvider } from './lib/Toast'
274 import { applyTheme, getInitialTheme } from './lib/theme'
275
276 // 在首帧渲染前应用主题，避免闪烁（HTML <head> 内联脚本会在 index.html 兜底）
277 applyTheme(getInitialTheme())
278
279 // 清理 PWA 时代残留的 Service Worker 与缓存，避免旧版页面被缓存拦截导致更新后仍显示旧内容
280 if ('serviceWorker' in navigator) {
281   navigator.serviceWorker.getRegistrations().then((registrations) => {
282     registrations.forEach((registration) => registration.unregister())
283   })
284 }
285 if ('caches' in window) {
286   caches.keys().then((keys) => {
287     keys.forEach((key) => caches.delete(key))
288   })
289 }
290
291 /** 视口宽度媒体查询，与 Tailwind 的 sm 断点（min-width: 640px）保持一致 */
292 function useMediaQuery(query: string): boolean {
293   const [matches, setMatches] = useState(() => typeof window !== 'undefined' && window.matchMe
    dia(query).matches)
294   useEffect(() => {
295     const mql = window.matchMedia(query)
296     const onChange = (e: MediaQueryListEvent) => setMatches(e.matches)
297     mql.addEventListener('change', onChange)
298     setMatches(mql.matches)
299     return () => mql.removeEventListener('change', onChange)
300   }, [query])
301   return matches
302 }
303
304 /**
305  * 网站桌面端布局：与电脑程序（Tauri「全部功能」模式）一致的「顶栏 + 左侧边栏」布局。
306  * 顶部栏不含窗口控制按钮，用「网页版」标识；内容区按高度自适应、首页双栏单屏。
307  */
308 function WebDesktopLayout() {
309   return (
310     <div className="theme-surface relative isolate flex h-screen w-full flex-col overflow-hidd
    en bg-gray-50 text-slate-900 dark:bg-slate-950 dark:text-slate-100">
311       <div className="flex items-center justify-between px-4 py-2.5 border-b border-gray-200 d
    ark:border-slate-800 shrink-0 select-none">
312         <div className="flex items-center gap-2">
313           <DesktopLogo size={22} />
314           <span className="text-sm font-semibold text-gray-700 dark:text-slate-300">DiveDeep</
    span>
```

```
315         </div>
316         <span className="text-xs text-gray-400 dark:text-slate-600">网页版</span>
317       </div>
318       <div className="flex-1 min-h-0 overflow-y-auto overscroll-contain">
319         <App hideBottomTab hideNavbar sidebar={<Sidebar />} fillHeight forceTwoCol />
320       </div>
321     </div>
322   )
323 }
324
325 /** 网站根组件：宽屏用电脑程序同款侧边栏布局，窄屏（手机/App）保留底部 Tab 移动布局 */
326 function WebRoot() {
327   const isDesktop = useMediaQuery('(min-width: 640px)')
328   return isDesktop ? <WebDesktopLayout /> : <App />
329 }
330
331 // Tauri 桌面版：渲染置顶计时小挂件；浏览器/移动端：渲染完整应用
332 if (isTauri()) {
333   // 挂件与全部功能模式共用同一主题（data-theme + dark class），保证两种模式外观一致
334   createRoot(document.getElementById('root')!).render(
335     <StrictMode>
336       <ToastProvider>
337         <AuthProvider>
338           <WidgetApp />
339         </AuthProvider>
340       </ToastProvider>
341     </StrictMode>,
342   )
343 } else {
344   createRoot(document.getElementById('root')!).render(
345     <StrictMode>
346       <WebRoot />
347     </StrictMode>,
348   )
349 }
350
351 // ==================== 文件: src/contexts/AuthContext.tsx ====================
352 import { createContext, useContext, useEffect, useState, type ReactNode } from 'react'
353 import { supabase, isSupabaseConfigured } from '../lib/supabase'
354 import { loadUserSubjects, resetSubjectCache } from '../lib/subjects'
355 import type { Session, User } from '@supabase/supabase-js'
356
357 interface AuthContextValue {
358   user: User | null
359   session: Session | null
360   loading: boolean
361   error: string | null
362   signUp: (email: string, password: string) => Promise<{ error: Error | null; needsEmailConfir
    mation: boolean }>
363   signIn: (email: string, password: string) => Promise<{ error: Error | null }>
364   signOut: () => Promise<void>
365 }
366
367 const AuthContext = createContext<AuthContextValue | undefined>(undefined)
368
```

```
export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null)
  const [session, setSession] = useState<Session | null>(null)
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState<string | null>(null)

  useEffect(() => {
    let cancelled = false

    // 环境变量未配置时，跳过网络请求，直接给出友好错误（避免请求占位域名导致超时）
    if (!isSupabaseConfigured) {
      setError('后端服务未配置，请检查部署环境变量设置')
      setLoading(false)
      return () => {
        cancelled = true
      }
    }

    // Get initial session
    supabase.auth.getSession().then(({ data: { session }, error: sessionError }) => {
      if (cancelled) return
      if (sessionError) {
        console.error('[Auth] getSession error:', sessionError.message)
        setError('认证服务连接失败，部分功能可能不可用')
      }
      setSession(session)
      setUser(session?.user ?? null)
      setLoading(false)
    }).catch((err) => {
      if (cancelled) return
      console.error('[Auth] getSession failed:', err)
      if (err instanceof DOMException && err.name === 'AbortError') {
        setError('认证服务连接超时，请检查网络后刷新重试')
      } else {
        setError('认证服务连接失败，部分功能可能不可用')
      }
      setLoading(false)
    })

    // Listen for auth state changes
    const { data: { subscription } } = supabase.auth.onAuthStateChange((_event, session) => {
      if (cancelled) return
      setSession(session)
      setUser(session?.user ?? null)
      setLoading(false)
      // 登录成功后加载该用户的云端自定义科目；登出时清空科目缓存
      if (session?.user) {
        loadUserSubjects(session.user.id)
      } else {
        resetSubjectCache()
      }
    })

    return () => {
      cancelled = true
```

```
      subscription.unsubscribe()
    }
  }, [])

  const signUp = async (email: string, password: string) => {
    const { data, error } = await supabase.auth.signUp({ email, password })
    // 如果 user 存在但 session 为空，说明需要邮箱确认
    const needsEmailConfirmation = !error && !!data.user && !data.session
    return { error, needsEmailConfirmation }
  }

  const signIn = async (email: string, password: string) => {
    const { error } = await supabase.auth.signInWithPassword({ email, password })
    return { error }
  }

  const signOut = async () => {
    await supabase.auth.signOut()
  }

  return (
    <AuthContext.Provider value={{ user, session, loading, error, signUp, signIn, signOut }}>
      {children}
    </AuthContext.Provider>
  )
}

export function useAuth(): AuthContextValue {
  const context = useContext(AuthContext)
  if (context === undefined) {
    throw new Error('useAuth must be used within an AuthProvider')
  }
  return context
}

// ==================== 文件: src/contexts/LogsContext.tsx ====================
import { createContext, useContext, useCallback, useEffect, useRef, useState, type ReactNode }
 from 'react'
import { useAuth } from './AuthContext'
import { fetchMyLogs, type DailyLog } from '../lib/dailyLogs'

interface LogsContextType {
  logs: DailyLog[]
  loading: boolean
  error: string | null
  refetch: () => void
}

const LogsContext = createContext<LogsContextType>({
  logs: [],
  loading: true,
  error: null,
  refetch: () => {},
})

```

```
export function LogsProvider({ children }: { children: ReactNode }) {
  const { user } = useAuth()
  const [logs, setLogs] = useState<DailyLog[]>([])
  const [loading, setLoading] = useState(true)
  const [error, setError] = useState<string | null>(null)
  const versionRef = useRef(0)

  const refetch = useCallback(() => {
    if (!user) return
    versionRef.current++
    setLoading(true)
    setError(null)
    const v = versionRef.current
    fetchMyLogs(user.id)
      .then((data) => {
        if (versionRef.current !== v) return
        setLogs(data)
      })
      .catch((err) => {
        if (versionRef.current !== v) return
        setError(err.message || '加载失败')
      })
      .finally(() => {
        if (versionRef.current !== v) return
        setLoading(false)
      })
  }, [user])

  useEffect(() => {
    if (!user) {
      setLogs([])
      setLoading(false)
      setError(null)
      return
    }
    refetch()
  }, [user, refetch])

  /* 回到前台/窗口重新聚焦时自动拉一次最新数据：
     其他设备修改后切回本端（或从后台返回）能立即看到最新记录，无需手动刷新 */
  useEffect(() => {
    if (!user) return
    const onVisibilityChange = () => {
      if (document.visibilityState === 'visible') refetch()
    }
    const onFocus = () => refetch()
    document.addEventListener('visibilitychange', onVisibilityChange)
    window.addEventListener('focus', onFocus)
    return () => {
      document.removeEventListener('visibilitychange', onVisibilityChange)
      window.removeEventListener('focus', onFocus)
    }
  }, [user, refetch])

  return (
```

```
    <LogsContext.Provider value={{ logs, loading, error, refetch }}>
      {children}
    </LogsContext.Provider>
  )
}

export function useLogs() {
  return useContext(LogsContext)
}

// ==================== 文件: src/contexts/UpdateContext.tsx ====================
import { createContext, useContext, type ReactNode } from 'react'
import { useAppUpdate, type UpdateStatus, type UpdateInfo } from '../hooks/useAppUpdate'

interface UpdateContextType {
  status: UpdateStatus
  updateInfo: UpdateInfo | null
  downloadProgress: number
  error: string | null
  checkForUpdate: () => Promise<UpdateInfo | null>
  downloadAndInstall: (info: UpdateInfo) => Promise<any>
}

const UpdateContext = createContext<UpdateContextType | null>(null)

/** 在 App 根组件包裹，实现启动时自动静默检查更新 */
export function UpdateProvider({ children }: { children: ReactNode }) {
  const update = useAppUpdate()
  return (
    <UpdateContext.Provider value={update}>
      {children}
    </UpdateContext.Provider>
  )
}

/** 在「我的」页面等子组件中使用，读取共享的更新状态 */
export function useUpdateContext() {
  const ctx = useContext(UpdateContext)
  if (!ctx) throw new Error('useUpdateContext 必须在 UpdateProvider 内使用')
  return ctx
}

// ==================== 文件: src/pages/Achievements.tsx ====================
import { useMemo } from 'react'
import { useLogs } from '../contexts/LogsContext'
import { Icon } from '../components/Icon'
import {
  ACHIEVEMENT_CATEGORIES,
  computeAchievements,
  computeStudyStats,
  type AchievementState,
  type AchievementCategory,
} from '../lib/achievements'
import { useWideLayout } from '../App'

```

```
/** 单个成就徽章 */
function Badge({ state }: { state: AchievementState }) {
  const { def, unlocked, current, progress } = state
  const displayCurrent = Math.min(current, def.target)
  const fmtNum = (n: number) => (Number.isInteger(n) ? String(n) : n.toFixed(1))
  return (
    <div
      className={`flex flex-col items-center gap-1 rounded-xl border p-3 text-center transitio
n-colors ${
        unlocked
          ? 'bg-amber-50 border-amber-200 dark:bg-amber-900/20 dark:border-amber-700/50'
          : 'bg-white dark:bg-slate-800 border-gray-100 dark:border-slate-700 opacity-60'
      }`}
      title={`${def.name}：${def.desc}（${fmtNum(displayCurrent)} / ${def.target}）`}
    >
      <span className={`text-2xl leading-none ${unlocked ? '' : 'grayscale'}`}>{def.icon}</spa
n>
      <span className={`text-sm font-semibold ${unlocked ? 'text-amber-700 dark:text-amber-300
' : 'text-gray-600 dark:text-slate-300'}`}>
        {def.name}
      </span>
      <span className="text-[11px] leading-tight text-gray-400 dark:text-slate-400 min-h-[2rem
]">{def.desc}</span>
      <div className="w-full h-1.5 bg-gray-100 dark:bg-slate-700 rounded-full overflow-hidden
mt-1">
        <div
          className={`h-full rounded-full transition-all ${unlocked ? 'bg-amber-400 dark:bg-am
ber-500' : 'bg-gray-300 dark:bg-slate-500'}`}
          style={{ width: `${progress}%` }}
        />
      </div>
      <span className={`text-[11px] tabular-nums ${unlocked ? 'text-amber-600 dark:text-amber-
300 font-medium' : 'text-gray-400 dark:text-slate-500'}`}>
        {unlocked ? '已解锁' : `${fmtNum(displayCurrent)} / ${def.target}`}
      </span>
    </div>
  )
}

export default function Achievements() {
  const wide = useWideLayout()
  const { logs, loading, error, refetch } = useLogs()

  const states = useMemo(() => computeAchievements(logs), [logs])
  const stats = useMemo(() => computeStudyStats(logs), [logs])
  const unlockedCount = states.filter((s) => s.unlocked).length

  const grouped = useMemo(() => {
    const map = new Map<AchievementCategory, AchievementState[]>()
    for (const state of states) {
      const list = map.get(state.def.category) ?? []
      list.push(state)
      map.set(state.def.category, list)
    }
    return map
```

```
636   }, [states])
637
638   return (
639     <div className={`mx-auto ${wide ? 'max-w-[1280px]' : 'max-w-4xl'} px-4 py-4 space-y-4`}>
640       <h1 className="flex items-center gap-2 text-xl font-bold text-gray-800 dark:text-slate-1
    00">
641         <Icon name="medal" size={20} className="text-amber-500" /> 成就
642       </h1>
643
644       {loading && (
645         <div className="flex justify-center py-12">
646           <div className="h-8 w-8 animate-spin rounded-full border-4 border-blue-200 dark:bord
    er-slate-700 border-t-blue-600 dark:border-t-blue-500" />
647         </div>
648       )}
649
650       {error && !loading && (
651         <div className="bg-white dark:bg-slate-800 rounded-xl shadow-sm border border-gray-100
     dark:border-slate-700 p-8 text-center">
652           <p className="text-red-500 dark:text-red-400">{error}</p>
653           <button
654             type="button"
655             onClick={refetch}
656             className="bg-blue-600 text-white hover:bg-blue-700 px-4 py-2 rounded-lg mt-3 tran
    sition-colors cursor-pointer"
657           >
658             重试
659           </button>
660         </div>
661       )}
662
663       {!loading && !error && (
664         <>
665           {/* 总览卡片 */}
666           <div className="grid grid-cols-2 sm:grid-cols-4 gap-3">
667             <div className="rounded-xl bg-white dark:bg-slate-800 p-4 shadow-sm border border-
    gray-100 dark:border-slate-700">
668               <p className="text-xs text-gray-500 dark:text-slate-400">已解锁</p>
669               <p className="text-2xl font-bold text-gray-900 dark:text-slate-100 mt-1">
670                 {unlockedCount}<span className="text-sm text-gray-400 font-normal"> / {states.
    length}</span>
671               </p>
672             </div>
673             <div className="rounded-xl bg-orange-50 dark:bg-orange-900/20 border-orange-200 da
    rk:border-orange-700/50 border p-4">
674               <p className="text-xs text-orange-600 dark:text-orange-300">当前连续</p>
675               <p className="text-2xl font-bold text-orange-600 dark:text-orange-300 mt-1">🔥 {
    stats.currentStreak} 天</p>
676             </div>
677             <div className="rounded-xl bg-blue-50 dark:bg-blue-900/20 border-blue-200 dark:bor
    der-blue-700/50 border p-4">
678               <p className="text-xs text-blue-600 dark:text-blue-300">最长连续</p>
679               <p className="text-2xl font-bold text-blue-600 dark:text-blue-300 mt-1">{stats.l
    ongestStreak} 天</p>
680             </div>
```

```
            <div className="rounded-xl bg-green-50 dark:bg-green-900/20 border-green-200 dark:
    border-green-700/50 border p-4">
              <p className="text-xs text-green-600 dark:text-green-300">累计学习</p>
              <p className="text-2xl font-bold text-green-600 dark:text-green-300 mt-1">
                {stats.totalHours.toFixed(1)}<span className="text-sm font-normal"> h</span>
              </p>
            </div>
          </div>

          {/* 分类徽章 */}
          {ACHIEVEMENT_CATEGORIES.map((cat) => {
            const list = grouped.get(cat.key) ?? []
            if (list.length === 0) return null
            const catUnlocked = list.filter((s) => s.unlocked).length
            return (
              <div key={cat.key} className="space-y-2">
                <div className="flex items-center gap-2 pt-2">
                  <span className="text-base">{cat.icon}</span>
                  <h2 className="text-base font-semibold text-gray-700 dark:text-slate-200">{c
    at.label}</h2>
                  <span className="text-xs text-gray-400 dark:text-slate-500">{catUnlocked} /
    {list.length}</span>
                </div>
                <div className="grid grid-cols-3 sm:grid-cols-4 md:grid-cols-6 gap-3">
                  {list.map((state) => (
                    <Badge key={state.def.id} state={state} />
                  ))}
                </div>
              </div>
            )
          })}
        </>
      )}
    </div>
  )
}

// ==================== 文件: src/pages/EditRecord.tsx ====================
import { useEffect, useRef, useState } from 'react'
import { useParams, useNavigate } from 'react-router-dom'
import { useAuth } from '../contexts/AuthContext'
import { useLogs } from '../contexts/LogsContext'
import { useToast } from '../lib/Toast'
import LogForm from '../components/LogForm'
import ConfirmDialog from '../components/ConfirmDialog'
import { getAvailableSubjects } from '../lib/subjects'
import type { DailyLog, DailyLogInput } from '../lib/dailyLogs'
import { fetchLogById, fetchLogByDate, updateLog, purgeLog, mergeSubjects, isDuplicateDateErro
    r } from '../lib/dailyLogs'
import { useWideLayout } from '../App'

/** 合并两段总结：都非空时换行拼接 */
function combineSummary(a: string, b: string): string {
  if (!a.trim()) return b
  if (!b.trim()) return a
```

```
732   return `${a.trim()}\n${b.trim()}`
733 }
734
735 export default function EditRecord() {
736   const wide = useWideLayout()
737   const { id } = useParams<{ id: string }>()
738   const { user } = useAuth()
739   const { refetch } = useLogs()
740   const toast = useToast()
741   const navigate = useNavigate()
742   const [log, setLog] = useState<DailyLog | null>(null)
743   const [loading, setLoading] = useState(true)
744   const [error, setError] = useState<string | null>(null)
745   const [mergeConfirm, setMergeConfirm] = useState<{ open: boolean; data?: DailyLogInput; targ
    et?: DailyLog }>({ open: false })
746   const pendingRef = useRef<{ data: DailyLogInput; target: DailyLog } | null>(null)
747
748   useEffect(() => {
749     if (!user || !id) return
750     setLoading(true)
751     fetchLogById(id)
752       .then((found) => {
753         if (!found) {
754           setError('记录不存在')
755         } else if (found.user_id !== user.id) {
756           setError('无权编辑此记录')
757         } else {
758           setLog(found)
759         }
760       })
761       .catch((err) => setError(err instanceof Error ? err.message : '加载失败'))
762       .finally(() => setLoading(false))
763   }, [user, id])
764
765   const doMerge = async (data: DailyLogInput, target: DailyLog) => {
766     if (!id) return
767     try {
768       await updateLog(target.id, {
769         date: data.date,
770         subjects: mergeSubjects(target.subjects, data.subjects),
771         summary: combineSummary(target.summary, data.summary),
772       })
773       await purgeLog(id)
774       refetch()
775       navigate('/my-records')
776     } catch (err) {
777       toast.show(err instanceof Error ? err.message : '保存失败，请重试', { icon: '❌' })
778     }
779   }
780
781   const handleSubmit = async (data: DailyLogInput) => {
782     if (!id || !user) return
783     try {
784       if (log && data.date !== log.date) {
785         const target = await fetchLogByDate(user.id, data.date)
```

```
        if (target) {
          pendingRef.current = { data, target }
          setMergeConfirm({ open: true, data, target })
          return
        }
      }
      await updateLog(id, data)
      refetch()
      navigate('/my-records')
    } catch (err) {
      if (isDuplicateDateError(err)) {
        toast.show('该日期已有记录，请选择其他日期', { icon: '⚠️' })
      } else {
        toast.show(err instanceof Error ? err.message : '保存失败，请重试', { icon: '❌' })
      }
    }
  }

  const handleMergeConfirm = () => {
    setMergeConfirm({ open: false })
    if (pendingRef.current) {
      doMerge(pendingRef.current.data, pendingRef.current.target)
      pendingRef.current = null
    }
  }

  const handleCancel = () => {
    navigate('/my-records')
  }

  if (loading) {
    return (
      <div className="flex justify-center py-12">
        <div className="h-8 w-8 animate-spin rounded-full border-4 border-blue-200 dark:border
-slate-700 border-t-blue-600 dark:border-t-blue-500" />
      </div>
    )
  }

  if (error || !log) {
    return (
      <div className="mx-auto max-w-2xl px-4 py-12 text-center">
        <p className="text-red-500 dark:text-red-400 mb-4">{error || '记录不存在'}</p>
        <button
          onClick={() => navigate('/my-records')}
          className="text-blue-600 dark:text-blue-400 hover:underline"
        >
          返回记录
        </button>
      </div>
    )
  }

  return (
    <div className={`mx-auto ${wide ? 'max-w-[1280px]' : 'max-w-2xl'} px-4 py-6`}>
```

```
      <h1 className="text-2xl font-bold text-gray-800 dark:text-slate-100 mb-6">编辑记录</h1>
      <LogForm
        initialData={{ date: log.date, subjects: log.subjects, summary: log.summary }}
        availableSubjects={getAvailableSubjects()}
        onSubmit={handleSubmit}
        onCancel={handleCancel}
      />
      <ConfirmDialog
        open={mergeConfirm.open}
        title="合并并删除原记录"
        message={`该日期（${mergeConfirm.target?.date}）已有记录。确认后将把当前编辑内容合并进该记录，并永久删除原记录（不可恢复）。`}
        confirmText="确认合并"
        danger
        onConfirm={handleMergeConfirm}
        onCancel={() => setMergeConfirm({ open: false })}
      />
    </div>
  )
}

// ==================== 文件: src/pages/EnglishCheckin.tsx ====================
import { useEffect, useMemo, useState, useCallback, Component } from 'react'
import type { ReactNode } from 'react'
import { useAuth } from '../contexts/AuthContext'
import { loadEnglishDaily, type EnglishDay } from '../data/englishDaily'
import { fetchMyCheckins, createCheckin, deleteCheckin } from '../lib/englishCheckin'
import { loadMarkedWords, saveMarkedWords, addDayToVocabulary, pushVocabularyToCloud } from '../lib/vocabulary'
import { supabase } from '../lib/supabase'
import { postJson } from '../lib/httpRequest'
import { useWideLayout } from '../App'

const TOTAL = 150

// ---------- AI 翻译批改 ----------

interface AiCorrection {
  score: number | null
  corrected: string
  issues: string[]
  suggestions: string[]
  backbone: string
  structure: string[]
  collocations: string[]
}

// 兜底归一化 AI 返回：强制所有字段为字符串/字符串数组/数字，杜绝畸形结构导致 React 渲染崩溃
function normalizeAiCorrection(raw: any): AiCorrection {
  const str = (v: unknown) => (typeof v === 'string' ? v : v == null ? '' : String(v))
  // 模型偶发把 issues/suggestions/structure 返回成对象列表（如 {问题, 建议}），
  // 这里提取首个字符串字段，避免前端渲染成 [object Object]
  const itemStr = (v: unknown): string => {
    if (typeof v === 'string') return v
    if (v && typeof v === 'object') {
```

```
893       const first = Object.values(v as Record<string, unknown>).find(x => typeof x === 'string
    ')
894       return first != null ? String(first) : JSON.stringify(v)
895     }
896     return v == null ? '' : String(v)
897   }
898   const strArr = (v: unknown) => (Array.isArray(v) ? v.map(itemStr).filter(Boolean) : [])
899   const num = (v: unknown) =>
900     typeof v === 'number' && Number.isFinite(v) ? v : v == null ? null : (Number(v) as number)
     || null
901   return {
902     score: num(raw?.score),
903     corrected: str(raw?.corrected),
904     issues: strArr(raw?.issues),
905     suggestions: strArr(raw?.suggestions),
906     backbone: str(raw?.backbone),
907     structure: strArr(raw?.structure),
908     collocations: strArr(raw?.collocations),
909   }
910 }
911
912 // 局部错误边界：AI 卡片渲染异常时只显示错误提示，避免整个页面崩溃变黑屏
913 interface EBState { hasError: boolean }
914 class AiResultBoundary extends Component<{ children: ReactNode }, EBState> {
915   state: EBState = { hasError: false }
916   static getDerivedStateFromError(): EBState { return { hasError: true } }
917   render() {
918     if (this.state.hasError) {
919       return <p className="text-xs text-red-500">AI 结果渲染异常，请重新点击解析</p>
920     }
921     return this.props.children
922   }
923 }
924
925 // CloudBase 云函数 HTTP 地址（国内节点直连讯飞，稳定）；未配置时回退到 Supabase Edge Function
926 const AI_CORRECT_URL = (import.meta.env.VITE_AI_CORRECT_URL as string | undefined)?.trim() ||
    ''
927
928 // ---------- 翻译相似度打分 ----------
929
930 /** 计算两个字符串的相似度，返回 0-100 的分数 */
931 function calcTranslationScore(userText: string, refText: string): number {
932   const u = userText.trim()
933   const r = refText.trim()
934   if (!u) return 0
935   if (u === r) return 100
936
937   // 1. 字符级相似度（最长公共子序列比率）
938   const lcsLen = lcs(u, r)
939   const charScore = (2 * lcsLen) / (u.length + r.length) * 100
940
941   // 2. 词级相似度（Jaccard）
942   const tokenize = (s: string) => {
943     const tokens: string[] = []
944     // 中文按字符拆，英文/数字按词拆
```

```
    const segs = s.split(/([\u4e00-\u9fff]+)/)
    for (const seg of segs) {
      if (/[\u4e00-\u9fff]/.test(seg)) {
        for (const ch of seg) tokens.push(ch)
      } else {
        tokens.push(...seg.split(/[^a-zA-Z0-9]+/).filter(Boolean).map(t => t.toLowerCase()))
      }
    }
    return new Set(tokens)
  }
  const uSet = tokenize(u)
  const rSet = tokenize(r)
  const intersection = new Set([...uSet].filter(x => rSet.has(x)))
  const union = new Set([...uSet, ...rSet])
  const wordScore = union.size > 0 ? (intersection.size / union.size) * 100 : 0

  // 综合：字符 60% + 词 40%
  return Math.round(charScore * 0.6 + wordScore * 0.4)
}

function lcs(a: string, b: string): number {
  const m = a.length, n = b.length
  if (m === 0 || n === 0) return 0
  // 使用滚动数组节省内存
  let prev = new Array(n + 1).fill(0)
  let cur = new Array(n + 1).fill(0)
  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      cur[j] = a[i - 1] === b[j - 1] ? prev[j - 1] + 1 : Math.max(prev[j], cur[j - 1])
    }
    ;[prev, cur] = [cur, prev]
  }
  return prev[n]
}

// ---------- 单词分词 ----------

interface Token {
  text: string
  isWord: boolean
  idx: number
}

/** 将英文句子拆分为可点击的单词 token */
function tokenizeSentence(text: string): Token[] {
  const tokens: Token[] = []
  let wordIdx = 0
  // 匹配单词（含缩写 don't, it's 和连字符词 co-operative, state-of-the-art）
  const re = /([a-zA-Z]+(?:[-'][a-zA-Z]+)*)|([^a-zA-Z]+)/g
  let m
  while ((m = re.exec(text)) !== null) {
    if (m[1]) {
      tokens.push({ text: m[1], isWord: true, idx: wordIdx++ })
    } else if (m[2]) {
      // 非单词字符，按字符拆分
```

```
      for (const ch of m[2]) {
        tokens.push({ text: ch, isWord: false, idx: -1 })
      }
    }
  }
  return tokens
}

// ---------- 长难句切分还原 ----------

/** 切分片段是无空格的粘连文本，对照原句 en 还原出空格，让单词可读。
 * 数据里可能存在三种噪音：① 单词粘连；② 讲解性括号注释（如 (that)、(at least)，
 * 括号或括号内的词在原句中并不存在）；③ 连字符/撇号/引号被 PDF 抽取改写（-- vs -、’ vs '）。
 * 这里通过「标点折叠 + 多候选匹配」尽可能把片段还原成原句中的连续子串。 */
function restoreChunkSpaces(chunk: string, orig: string): string {
  if (!chunk) return chunk
  const needleRaw = chunk.replace(/\s+/g, '')
  if (!needleRaw) return chunk

  // 原句去掉空格，并记录「去空格下标 -> 原句下标」映射
  const origMap: number[] = []
  let strippedBuilder = ''
  for (let i = 0; i < orig.length; i++) {
    if (/\s/.test(orig[i])) continue
    strippedBuilder += orig[i]
    origMap.push(i)
  }

  // 标点折叠：连字符/破折号统一为 '-'（连续多个合并成一个）、撇号统一为 "'"、引号统一为 '"'
  const foldChar = (c: string): { out: string; skip: boolean } => {
    if (/[\u002D\u2010-\u2015]/.test(c)) return { out: '-', skip: false }
    if (/[\u0027\u2018\u2019\u201A\u201B]/.test(c)) return { out: "'", skip: false }
    if (/[\u0022\u201C\u201D\u201E\u201F]/.test(c)) return { out: '"', skip: false }
    return { out: c, skip: false }
  }

  // 从「去空格字符串」折叠，同时记录折叠下标 -> 去空格下标
  let folded = ''
  const foldMap: number[] = []
  let prevDash = false
  for (let i = 0; i < strippedBuilder.length; i++) {
    const { out, skip } = foldChar(strippedBuilder[i])
    if (skip) continue
    if (out === '-') {
      if (prevDash) continue // 合并连续连字符
      prevDash = true
      folded += '-'
      foldMap.push(i)
      continue
    }
    prevDash = false
    folded += out
    foldMap.push(i)
  }
```

```
1054
1055   // 片段同样折叠
1056   const foldChunk = (s: string): string =>
1057     s
1058       .replace(/[\u002D\u2010-\u2015]+/g, '-')
1059       .replace(/[\u0027\u2018\u2019\u201A\u201B]+/g, "'")
1060       .replace(/[\u0022\u201C\u201D\u201E\u201F]+/g, '"')
1061
1062   const F = foldChunk(needleRaw)
1063   // 候选片段（按优先级生成，最终取「最长命中」以保留最多原文信息）：
1064   // 1) 原样 2) 去掉括号保留内容 3) 去掉整组括号 4) 先整组再去残留括号
1065   const candidates = [
1066     F,
1067     F.replace(/[()]/g, ''),
1068     F.replace(/\([^()]*\)/g, ''),
1069     F.replace(/\([^()]*\)/g, '').replace(/[()]/g, ''),
1070   ]
1071
1072   let bestStart = -1
1073   let bestLen = 0
1074   for (const cand of candidates) {
1075     if (!cand) continue
1076     const idx = folded.indexOf(cand)
1077     if (idx !== -1 && cand.length > bestLen) {
1078       bestStart = idx
1079       bestLen = cand.length
1080     }
1081   }
1082
1083   if (bestStart === -1) return chunk // 无法匹配则原样显示
1084   const startOrig = origMap[foldMap[bestStart]]
1085   const endOrig = origMap[foldMap[bestStart + bestLen - 1]]
1086   return orig.slice(startOrig, endOrig + 1)
1087 }
1088
1089 // ---------- 颜色工具 ----------
1090
1091 function scoreColor(score: number): string {
1092   if (score >= 80) return 'text-green-600 dark:text-green-400'
1093   if (score >= 60) return 'text-yellow-600 dark:text-yellow-400'
1094   return 'text-red-600 dark:text-red-400'
1095 }
1096
1097 function scoreBg(score: number): string {
1098   if (score >= 80) return 'bg-green-100 dark:bg-green-900/30'
1099   if (score >= 60) return 'bg-yellow-100 dark:bg-yellow-900/30'
1100   return 'bg-red-100 dark:bg-red-900/30'
1101 }
1102
1103 // ---------- 页面组件 ----------
1104
1105 export default function EnglishCheckin() {
1106   const wide = useWideLayout()
1107   const { user } = useAuth()
1108   const [checkins, setCheckins] = useState<Set<number>>(new Set())
```

```
  const [loading, setLoading] = useState(true)
  const [selectedDay, setSelectedDay] = useState(1)
  const [busy, setBusy] = useState(false)
  const [error, setError] = useState<string | null>(null)

  // 标记单词：Set<"dayIdx-sentIdx-wordIdx">，从 localStorage 初始化
  const [markedWords, setMarkedWords] = useState<Set<string>>(() => loadMarkedWords())
  // 翻译输入：Map<"dayIdx-sentIdx", string>
  const [translations, setTranslations] = useState<Map<string, string>>(new Map())
  // 打分结果：Map<"dayIdx-sentIdx", number | null>
  const [scores, setScores] = useState<Map<string, number | null>>(new Map())
  // AI 批改结果：Map<"dayIdx-sentIdx", AiCorrection | null>
  const [aiResults, setAiResults] = useState<Map<string, AiCorrection | null>>(new Map())
  // AI 批改进行中的句子：Set<"dayIdx-sentIdx">
  const [aiLoading, setAiLoading] = useState<Set<string>>(new Set())
  // AI 批改错误信息
  const [aiError, setAiError] = useState<string | null>(null)
  // 预生成「解析」卡片展开状态：Set<"dayIdx-sentIdx">
  const [showAnalysis, setShowAnalysis] = useState<Set<string>>(new Set())
  // 150 天语料（懒加载 JSON chunk，含原文/译文/解析）
  const [daily, setDaily] = useState<EnglishDay[]>([])

  const completedDays = useMemo(() => new Set(checkins), [checkins])
  const nextDay = useMemo(() => {
    for (let d = 1; d <= TOTAL; d++) if (!completedDays.has(d)) return d
    return TOTAL + 1
  }, [completedDays])
  const lastDoneDay = nextDay - 1
  const progress = (completedDays.size / TOTAL) * 100
  const allDone = nextDay > TOTAL

  useEffect(() => {
    if (!user) { setLoading(false); return }
    setLoading(true)
    fetchMyCheckins(user.id)
      .then((list) => {
        const set = new Set(list.map((c) => c.day))
        setCheckins(set)
        for (let d = 1; d <= TOTAL; d++) {
          if (!set.has(d)) { setSelectedDay(d); break }
        }
      })
      .catch((e) => setError(e instanceof Error ? e.message : '加载失败'))
      .finally(() => setLoading(false))
  }, [user])

  // 语料懒加载（独立异步 chunk，不阻塞首屏其它逻辑）
  useEffect(() => {
    let cancelled = false
    loadEnglishDaily()
      .then((d) => { if (!cancelled) setDaily(d) })
      .catch((e) => setError(e instanceof Error ? e.message : '语料加载失败'))
    return () => { cancelled = true }
  }, [])

```

```
  const dayData: EnglishDay | undefined = daily.find((d) => d.day === selectedDay) ?? daily[0]
  const isDone = completedDays.has(selectedDay)
  const isNext = selectedDay === nextDay

  const handleCheckIn = async () => {
    if (!user || isDone || nextDay > TOTAL || busy) return
    setBusy(true); setError(null)
    try {
      await createCheckin(user.id, nextDay)

      // 收集当天标记的生词，存入生词本
      const dayVocabWords: { word: string; sentence: string; sentIdx: number; wordIdx: number
}[] = []
      const dayData = daily.find((d) => d.day === nextDay)
      if (dayData) {
        for (let si = 0; si < dayData.sentences.length; si++) {
          const tokens = tokenizeSentence(dayData.sentences[si].en)
          for (const t of tokens) {
            if (t.isWord && markedWords.has(`${nextDay}-${si}-${t.idx}`)) {
              dayVocabWords.push({ word: t.text, sentence: dayData.sentences[si].en, sentIdx:
si, wordIdx: t.idx })
            }
          }
        }
        if (dayVocabWords.length > 0) {
          addDayToVocabulary(nextDay, dayVocabWords)
          // 同步推送到云端生词本（登录用户）
          try {
            await pushVocabularyToCloud(
              user.id,
              dayVocabWords.map((w) => ({ ...w, day: nextDay })),
            )
          } catch { /* 忽略同步失败，本地已保存 */ }
        }
      }

      const set = new Set(checkins); set.add(nextDay); setCheckins(set)
      setSelectedDay(nextDay + 1 > TOTAL ? TOTAL : nextDay + 1)
    } catch (e) {
      setError(e instanceof Error ? e.message : '打卡失败')
    } finally { setBusy(false) }
  }

  const handleUndo = async () => {
    if (!user || lastDoneDay < 1 || busy) return
    setBusy(true); setError(null)
    try {
      await deleteCheckin(user.id, lastDoneDay)
      const set = new Set(checkins); set.delete(lastDoneDay); setCheckins(set)
      setSelectedDay(lastDoneDay)
    } catch (e) {
      setError(e instanceof Error ? e.message : '撤销失败')
    } finally { setBusy(false) }
  }

```

```
1217  const toggleWord = useCallback((dayIdx: number, sentIdx: number, wordIdx: number) => {
1218    const key = `${dayIdx}-${sentIdx}-${wordIdx}`
1219    setMarkedWords(prev => {
1220      const next = new Set(prev)
1221      if (next.has(key)) next.delete(key)
1222      else next.add(key)
1223      saveMarkedWords(next)
1224      return next
1225    })
1226  }, [])
1227
1228  const handleScore = useCallback((dayIdx: number, sentIdx: number) => {
1229    const key = `${dayIdx}-${sentIdx}`
1230    const userText = translations.get(key) || ''
1231    const day = daily.find(d => d.day === dayIdx)
1232    const refText = day?.sentences[sentIdx]?.ref || ''
1233    const score = calcTranslationScore(userText, refText)
1234    setScores(prev => { const next = new Map(prev); next.set(key, score); return next })
1235  }, [translations, daily])
1236
1237  // 展开/收起预生成「解析」卡片（本地数据，不消耗 AI token）
1238  const toggleAnalysis = useCallback((dayIdx: number, sentIdx: number) => {
1239    const key = `${dayIdx}-${sentIdx}`
1240    setShowAnalysis(prev => {
1241      const next = new Set(prev)
1242      if (next.has(key)) next.delete(key)
1243      else next.add(key)
1244      return next
1245    })
1246  }, [])
1247
1248  // 调用 Supabase Edge Function 完成 AI 翻译批改
1249  const handleAiCorrect = useCallback(async (dayIdx: number, sentIdx: number) => {
1250    const key = `${dayIdx}-${sentIdx}`
1251    if (aiLoading.has(key)) return
1252    const userText = translations.get(key) || ''
1253    const day = daily.find(d => d.day === dayIdx)
1254    const sentence = day?.sentences[sentIdx]?.en || ''
1255    // 参考译文只取当前这一句的（来自逐句译文 ref），不要传整段译文，
1256    // 否则模型会把"修正译文"直接输出成整段参考译文
1257    const refText = day?.sentences[sentIdx]?.ref || ''
1258
1259    setAiLoading(prev => new Set(prev).add(key))
1260    setAiError(null)
1261    try {
1262      let result: AiCorrection
1263      if (AI_CORRECT_URL) {
1264        // 走腾讯云 SCF 云函数；原生平台用原生网络栈，绕开 WebView fetch 崩溃黑屏
1265        const { ok, status, body } = await postJson(AI_CORRECT_URL, {
1266          en: sentence,
1267          userTranslation: userText,
1268          refTranslation: refText,
1269        })
1270        if (!ok || !body?.ok) throw new Error(body?.error || `AI 批改失败（${status}）`)
1271        result = normalizeAiCorrection(body.data)
```

```
1272      } else {
1273        // 回退：Supabase Edge Function
1274        const { data, error } = await supabase.functions.invoke('ai-correct', {
1275          body: { en: sentence, userTranslation: userText, refTranslation: refText },
1276        })
1277        if (error) throw new Error(error.message || 'AI 批改失败')
1278        result = normalizeAiCorrection(data)
1279      }
1280      setAiResults(prev => { const next = new Map(prev); next.set(key, result); return next })
1281    } catch (e) {
1282      const aborted = e instanceof Error && e.name === 'AbortError'
1283      setAiError(
1284        aborted
1285          ? 'AI 批改超时，请检查网络后重试'
1286          : e instanceof Error ? e.message : 'AI 批改失败，请稍后重试'
1287      )
1288    } finally {
1289      setAiLoading(prev => { const next = new Set(prev); next.delete(key); return next })
1290    }
1291  }, [translations, aiLoading, daily])
1292
1293  // 语料未就绪的兜底（必须放在最后一个 Hooks 之后，避免违反 Hooks 规则）
1294  if (!dayData) {
1295    return (
1296      <div className="mx-auto max-w-5xl px-4 py-4">
1297        <div className="rounded-xl bg-white dark:bg-slate-800 p-8 shadow-sm border border-gray
     -100 dark:border-slate-700 text-center">
1298          <p className="text-sm text-gray-500 dark:text-slate-400">
1299            {error ? `语料加载失败：${error}` : '语料加载中…'}
1300          </p>
1301        </div>
1302      </div>
1303    )
1304  }
1305
1306  return (
1307    <div className={`mx-auto ${wide ? 'max-w-[1280px]' : 'max-w-5xl'} px-4 py-4 space-y-4`}>
1308      {/* 进度条 */}
1309      <div className="rounded-xl bg-white dark:bg-slate-800 p-4 shadow-sm border border-gray-1
     00 dark:border-slate-700">
1310        <div className="flex items-center justify-between">
1311          <h1 className="text-base font-bold">英语长难句打卡</h1>
1312          <span className="text-sm text-gray-500 dark:text-slate-400">{completedDays.size}/{TO
     TAL} 天</span>
1313        </div>
1314        <div className="w-full bg-gray-100 dark:bg-slate-700 rounded-full h-2.5 overflow-hidde
     n mt-3">
1315          <div className={`h-full rounded-full transition-all ${progress >= 100 ? 'bg-green-50
     0' : 'bg-blue-500'}`}
1316            style={{ width: `${progress}%` }} />
1317        </div>
1318        <p className="mt-2 text-xs text-gray-500 dark:text-slate-400">
1319          {allDone ? '🎉 恭喜完成全部 150 天打卡！' : `按顺序打卡，接下来是 Day ${nextDay}`}
1320        </p>
1321      </div>
```

```
1322
1323      {error && (
1324        <div className="rounded-xl bg-red-50 dark:bg-red-900/30 border border-red-200 dark:bor
     der-red-700/50 px-3 py-2 text-sm text-red-600 dark:text-red-300">{error}</div>
1325      )}
1326
1327      {aiError && (
1328        <div className="rounded-xl bg-amber-50 dark:bg-amber-900/30 border border-amber-200 da
     rk:border-amber-700/50 px-3 py-2 text-sm text-amber-700 dark:text-amber-300">AI 批改失败：{aiError}<
     /div>
1329      )}
1330
1331      {/* 逐句翻译：原文 + 翻译输入框放在一起 */}
1332      <div className="rounded-xl bg-white dark:bg-slate-800 p-4 shadow-sm border border-gray-1
     00 dark:border-slate-700">
1333        <h2 className="text-lg font-bold mb-1">Day {dayData.day}</h2>
1334        <div className="flex items-center gap-2 mb-3">
1335          <span className="px-2 py-0.5 rounded-full bg-green-100 text-green-800 dark:bg-green-
     900/40 dark:text-green-300 text-xs">{dayData.type}</span>
1336          <span className="px-2 py-0.5 rounded-full bg-gray-100 text-gray-600 dark:bg-slate-70
     0 dark:text-slate-300 text-xs">{dayData.source}</span>
1337          {isDone && <span className="px-2 py-0.5 rounded-full bg-blue-100 text-blue-800 dark:
     bg-blue-900/40 dark:text-blue-300 text-xs">已打卡</span>}
1338        </div>
1339        <p className="text-xs text-gray-400 dark:text-slate-500 mb-3">点击单词标记生词 · 逐句
     翻译后点击「打分」</p>
1340
1341        <div className="space-y-5">
1342          {dayData.sentences.map((s, sentIdx) => {
1343            const tokens = tokenizeSentence(s.en)
1344            const tKey = `${dayData.day}-${sentIdx}`
1345            const scoreVal = scores.get(tKey)
1346            const markedCount = tokens.filter(t => t.isWord && markedWords.has(`${dayData.day}
     -${sentIdx}-${t.idx}`)).length
1347
1348            return (
1349              <div key={sentIdx} className="border border-gray-100 dark:border-slate-700 round
     ed-lg p-3">
1350                {/* 原文 */}
1351                <div className="flex items-center justify-between mb-2">
1352                  <span className="text-xs font-medium text-green-600 dark:text-green-400">{s.
     num}</span>
1353                  {scoreVal != null && (
1354                    <span className={`text-xs font-bold px-2 py-0.5 rounded-full ${scoreBg(sco
     reVal)} ${scoreColor(scoreVal)}`}>
1355                      相似度 {scoreVal}%
1356                    </span>
1357                  )}
1358                </div>
1359                <p className="text-[15px] leading-relaxed text-gray-800 dark:text-slate-100 mb
     -2 select-none">
1360                  {tokens.map((t, i) =>
1361                    t.isWord ? (
1362                      <span
1363                        key={i}
```

```
                        onClick={() => toggleWord(dayData.day, sentIdx, t.idx)}
                        className={`cursor-pointer transition-colors rounded px-0.5 ${
                          markedWords.has(`${dayData.day}-${sentIdx}-${t.idx}`)
                            ? 'bg-red-200 text-red-700 dark:bg-red-900/50 dark:text-red-400 fo
nt-medium'
                            : 'hover:bg-gray-200 dark:hover:bg-slate-600'
                        }`}
                      >{t.text}</span>
                    ) : (
                      <span key={i}>{t.text}</span>
                    )
                  )}
                </p>
                {markedCount > 0 && (
                  <p className="text-xs text-red-500 dark:text-red-400 mb-2">{markedCount} 个
生词已标记</p>
                )}

                {/* 翻译输入 */}
                <textarea
                  value={translations.get(tKey) || ''}
                  onChange={e => {
                    const v = e.target.value
                    setTranslations(prev => { const next = new Map(prev); next.set(tKey, v); r
eturn next })
                    setScores(prev => { const next = new Map(prev); next.delete(tKey); return
next })
                    setAiResults(prev => { const next = new Map(prev); next.delete(tKey); retu
rn next })
                  }}
                  placeholder="在此输入你的翻译..."
                  rows={2}
                  className="w-full rounded-lg border border-gray-200 dark:border-slate-600 bg
-gray-50 dark:bg-slate-700/50 px-3 py-2 text-sm text-gray-800 dark:text-slate-100 placeholder-gray-4
00 dark:placeholder-slate-500 resize-none focus:outline-none focus:ring-2 focus:ring-green-400 focus
:border-transparent"
                />
                <div className="mt-1.5 flex flex-wrap items-center gap-2">
                  <button
                    onClick={() => handleScore(dayData.day, sentIdx)}
                    disabled={!translations.get(tKey)?.trim()}
                    className="px-3 py-1 rounded-lg bg-blue-500 hover:bg-blue-600 disabled:bg-
gray-300 dark:disabled:bg-slate-700 text-white text-xs font-medium transition-colors cursor-pointer
disabled:cursor-not-allowed"
                  >
                    打分
                  </button>
                  <button
                    onClick={() => toggleAnalysis(dayData.day, sentIdx)}
                    disabled={!s.ai}
                    title={s.ai ? '查看句子主干/结构/搭配解析' : '该句暂无预生成解析'}
                    className="px-3 py-1 rounded-lg bg-emerald-500 hover:bg-emerald-600 disabl
ed:bg-gray-300 dark:disabled:bg-slate-700 text-white text-xs font-medium transition-colors cursor-po
inter disabled:cursor-not-allowed"
                  >
```

```
1407                    解析
1408                  </button>
1409                  <button
1410                    onClick={() => handleAiCorrect(dayData.day, sentIdx)}
1411                    disabled={!translations.get(tKey)?.trim() || aiLoading.has(tKey)}
1412                    title="AI 批改会消耗 AI 调用额度，仅在需要时点击"
1413                    className="px-3 py-1 rounded-lg bg-purple-500 hover:bg-purple-600 disabled
    :bg-gray-300 dark:disabled:bg-slate-700 text-white text-xs font-medium transition-colors cursor-poin
    ter disabled:cursor-not-allowed flex items-center gap-1"
1414                  >
1415                    {aiLoading.has(tKey) ? (
1416                      <>
1417                        <span className="inline-block w-3 h-3 border-2 border-white/50 border-
    t-white rounded-full animate-spin" />
1418                        批改中...
1419                      </>
1420                    ) : (
1421                      <>AI 批改</>
1422                    )}
1423                  </button>
1424                </div>
1425
1426                {/* 预生成解析（本地数据，不消耗 AI token） */}
1427                {showAnalysis.has(tKey) && s.ai && (
1428                  <div className="mt-2 rounded-lg bg-emerald-50 dark:bg-emerald-900/20 border
    border-emerald-100 dark:border-emerald-800/40 p-3 space-y-2">
1429                    <div className="flex items-center justify-between">
1430                      <span className="text-xs font-bold text-emerald-700 dark:text-emerald-30
    0">句子解析</span>
1431                    </div>
1432                    {s.ai.backbone && (
1433                      <div className="rounded-md border border-emerald-200 dark:border-emerald
    -700/50 bg-white/60 dark:bg-slate-900/40 px-3 py-2">
1434                        <p className="text-xs font-bold text-emerald-700 dark:text-emerald-300
     mb-1">句子主干</p>
1435                        <p className="text-sm leading-relaxed text-gray-800 dark:text-slate-10
    0 break-words">{s.ai.backbone}</p>
1436                      </div>
1437                    )}
1438                    {s.ai.structure.length > 0 && (
1439                      <div>
1440                        <p className="text-xs font-medium text-emerald-600 dark:text-emerald-4
    00 mb-1">结构解析</p>
1441                        <ul className="list-disc list-inside space-y-0.5">
1442                          {s.ai.structure.map((item, i) => (
1443                            <li key={i} className="text-xs text-gray-700 dark:text-slate-200 b
    reak-words">{item}</li>
1444                          ))}
1445                        </ul>
1446                      </div>
1447                    )}
1448                    {s.ai.collocations.length > 0 && (
1449                      <div>
1450                        <p className="text-xs font-medium text-emerald-600 dark:text-emerald-4
    00 mb-1">短语搭配</p>
```

```
1451                        <ul className="list-disc list-inside space-y-0.5">
1452                          {s.ai.collocations.map((item, i) => (
1453                            <li key={i} className="text-xs text-gray-700 dark:text-slate-200 b
     reak-words">{item}</li>
1454                          ))}
1455                        </ul>
1456                      </div>
1457                    )}
1458                  </div>
1459                )}
1460
1461                {/* AI 批改结果 */}
1462                {aiResults.get(tKey) && (
1463                  <AiResultBoundary>
1464                    {(() => {
1465                  const r = aiResults.get(tKey)!
1466                  return (
1467                    <div className="mt-2 rounded-lg bg-purple-50 dark:bg-purple-900/20 border
     border-purple-100 dark:border-purple-800/40 p-3 space-y-2">
1468                      <div className="flex items-center justify-between">
1469                        <span className="text-xs font-bold text-purple-700 dark:text-purple-30
     0">AI 批改</span>
1470                        {r.score != null && (
1471                          <span className={`text-xs font-bold px-2 py-0.5 rounded-full ${score
     Bg(r.score)} ${scoreColor(r.score)}`}>
1472                            AI 评分 {r.score}
1473                          </span>
1474                        )}
1475                      </div>
1476                      {r.corrected && (
1477                        <div>
1478                          <p className="text-xs font-medium text-gray-500 dark:text-slate-400
     mb-1">修正译文</p>
1479                          <p className="text-sm leading-relaxed text-gray-800 dark:text-slate-
     100 break-words">{r.corrected}</p>
1480                        </div>
1481                      )}
1482                      {r.issues.length > 0 && (
1483                        <div>
1484                          <p className="text-xs font-medium text-red-500 dark:text-red-400 mb-
     1">存在的问题</p>
1485                          <ul className="list-disc list-inside space-y-0.5">
1486                            {r.issues.map((issue, i) => (
1487                              <li key={i} className="text-xs text-gray-700 dark:text-slate-200
      break-words">{issue}</li>
1488                            ))}
1489                          </ul>
1490                        </div>
1491                      )}
1492                      {r.suggestions.length > 0 && (
1493                        <div>
1494                          <p className="text-xs font-medium text-purple-600 dark:text-purple-4
     00 mb-1">改进建议</p>
1495                          <ul className="list-disc list-inside space-y-0.5">
1496                            {r.suggestions.map((sug, i) => (
```

```
1497                              <li key={i} className="text-xs text-gray-700 dark:text-slate-200
 break-words">{sug}</li>
1498                            ))}
1499                          </ul>
1500                        </div>
1501                      )}
1502                    </div>
1503                  )
1504                    })()}
1505                  </AiResultBoundary>
1506                )}
1507
1508                {/* 长难句解析 */}
1509                {dayData.analysis && (() => {
1510                  const aItem = dayData.analysis.find(a => a.sentNum === s.num)
1511                  if (!aItem) return null
1512                  const hasContent = aItem.vocab.length > 0 || aItem.split || aItem.grammar.le
ngth > 0 || aItem.ref
1513                  if (!hasContent) return null
1514                  return (
1515                    <details className="mt-2 group">
1516                      <summary className="cursor-pointer text-xs text-purple-600 dark:text-pur
ple-400 hover:text-purple-800 dark:hover:text-purple-300 select-none font-medium">
1517                        长难句解析
1518                      </summary>
1519                      <div className="mt-2 space-y-2 pl-1">
1520                        {aItem.vocab.length > 0 && (
1521                          <div>
1522                            <p className="text-xs font-medium text-gray-500 dark:text-slate-40
0 mb-1">单词</p>
1523                            <div className="flex flex-wrap gap-1">
1524                              {aItem.vocab.map((v, vi) => (
1525                                <span key={vi} className="inline-flex items-center gap-1 px-1.
5 py-0.5 rounded bg-purple-50 dark:bg-purple-900/20 text-xs">
1526                                  <span className="font-medium text-purple-700 dark:text-purpl
e-300">{v.word}</span>
1527                                  <span className="text-gray-500 dark:text-slate-400">{v.meani
ng}</span>
1528                                </span>
1529                              ))}
1530                            </div>
1531                          </div>
1532                        )}
1533                        {aItem.split && (() => {
1534                          // 用 / 分隔各切分段，并对照原句还原空格使单词可读
1535                          const parts = aItem.split.split('//').map((p) => p.trim())
1536                          const restored = parts.map((p) => restoreChunkSpaces(p, s.en))
1537                          return (
1538                            <div>
1539                              <p className="text-xs font-medium text-gray-500 dark:text-slate-
400 mb-1">切分</p>
1540                              <p className="text-xs leading-relaxed text-gray-600 dark:text-sl
ate-300 font-mono break-words">
1541                                {restored.map((part, pi) => (
1542                                  <span key={pi}>
```

```
      .select('*')
      .eq('user_id', userId)

    if (error) throw new Error(error.message)
    userSubjectsCache = (data as Array<{
      id: string
      name: string
      category?: string
      activities?: unknown
    }>).map((s) => ({
      id: s.id,
      name: s.name,
      category: s.category ?? 'custom',
      activities: Array.isArray(s.activities)
        ? (s.activities as string[]).filter((a): a is string => typeof a === 'string')
        : [],
    }))
    return true
  } catch (err) {
    userSubjectsError = err instanceof Error ? err.message : '未知错误'
    userSubjectsCache = []
    return false
  }
}

export function getAvailableSubjects(): Subject[] {
  const builtin: Subject[] = ALL_SUBJECTS
  const custom: SubjectWithActivities[] =
    userSubjectsCache && !userSubjectsError
      ? userSubjectsCache.filter((s) => !builtin.some((b) => b.id === s.id))
      : []
  return [...builtin, ...custom]
}

export function getSubjectById(id: string): Subject | undefined {
  if (!id) return undefined
  const builtin: Subject[] = ALL_SUBJECTS
  const custom: SubjectWithActivities[] = userSubjectsCache ?? []
  return builtin.find((s) => s.id === id) ?? custom.find((s) => s.id === id)
}

/** 按科目 category 返回可选学习内容；无配置（含自定义字符串内容）原样返回 */
export function getActivitiesForSubject(subjectId: string): string[] {
  const subject = getSubjectById(subjectId)
  if (!subject) return []
  const builtin = SUBJECT_ACTIVITIES[subject.category]
  if (builtin) return builtin
  const custom = userSubjectsCache?.find((s) => s.id === subjectId)
  return custom?.activities ?? []
}

/* ── 云端自定义科目增删改 ── */

export interface UserSubject {
  id: string
```

```
  user_id: string
  name: string
  category: string
  activities: string[]
}

/** 自定义科目在当前内置科目中已存在则视为内置（避免删除/改名内置科目） */
function isBuiltinId(id: string): boolean {
  return ALL_SUBJECTS.some((s) => s.id === id)
}

export async function fetchUserSubjects(userId: string): Promise<UserSubject[]> {
  const { data, error } = await supabase
    .from('user_subjects')
    .select('*')
    .eq('user_id', userId)

  if (error) throw new Error(error.message)
  return (data as Array<{ id: string; user_id: string; name: string; category: string; activities: unknown }>).map(
    (s) => ({
      id: s.id,
      user_id: s.user_id,
      name: s.name,
      category: s.category ?? 'custom',
      activities: Array.isArray(s.activities)
        ? (s.activities as string[]).filter((a): a is string => typeof a === 'string')
        : [],
    }),
  )
}

export async function createUserSubject(
  userId: string,
  input: { name: string; activities: string[] },
): Promise<UserSubject> {
  const { data, error } = await supabase
    .from('user_subjects')
    .insert({
      user_id: userId,
      name: input.name.trim(),
      category: 'custom',
      activities: input.activities.filter((a) => a.trim()).map((a) => a.trim()),
    })
    .select()
    .single()

  if (error) throw new Error(error.message)
  return data as UserSubject
}

export async function updateUserSubject(
  subjectId: string,
  input: { name: string; activities: string[] },
): Promise<void> {
```

```
14304   const { error } = await supabase
14305     .from('user_subjects')
14306     .update({
14307       name: input.name.trim(),
14308       activities: input.activities.filter((a) => a.trim()).map((a) => a.trim()),
14309     })
14310     .eq('id', subjectId)
14311
14312   if (error) throw new Error(error.message)
14313 }
14314
14315 export async function deleteUserSubject(subjectId: string): Promise<void> {
14316   const { error } = await supabase
14317     .from('user_subjects')
14318     .delete()
14319     .eq('id', subjectId)
14320
14321   if (error) throw new Error(error.message)
14322 }
14323
14324 /** 判断 id 是否为内置科目（内置科目不可删改） */
14325 export function isBuiltinSubject(subjectId: string): boolean {
14326   return isBuiltinId(subjectId)
14327 }
14328
14329 // ==================== 文件: src/lib/summary.ts ====================
14330 import { format, startOfWeek, endOfWeek, startOfMonth, endOfMonth, parseISO, differenceInCalen
      darDays } from 'date-fns'
14331 import type { DailyLog, DailyLogSubject } from './dailyLogs'
14332 import { getSubjectById } from './subjects'
14333
14334 export interface SummaryRange {
14335   mode: 'week' | 'month' | 'custom'
14336   startDate: string
14337   endDate: string
14338 }
14339
14340 export interface SubjectBreakdown {
14341   subjectId: string
14342   name: string
14343   hours: number
14344   percentage: number
14345 }
14346
14347 export interface DailyTrendItem {
14348   date: string
14349   totalHours: number
14350   subjects: DailyLogSubject[]
14351 }
14352
14353 export interface SummaryResult {
14354   totalHours: number
14355   checkedDays: number
14356   subjectBreakdown: SubjectBreakdown[]
14357   dailyTrend: DailyTrendItem[]
```

```
14358 }
14359
14360 export function getWeekRange(refDate: Date = new Date()): { startDate: string; endDate: string
       } {
14361   const start = startOfWeek(refDate, { weekStartsOn: 1 })
14362   const end = endOfWeek(refDate, { weekStartsOn: 1 })
14363   return {
14364     startDate: format(start, 'yyyy-MM-dd'),
14365     endDate: format(end, 'yyyy-MM-dd'),
14366   }
14367 }
14368
14369 export function getMonthRange(refDate: Date = new Date()): { startDate: string; endDate: strin
      g } {
14370   const start = startOfMonth(refDate)
14371   const end = endOfMonth(refDate)
14372   return {
14373     startDate: format(start, 'yyyy-MM-dd'),
14374     endDate: format(end, 'yyyy-MM-dd'),
14375   }
14376 }
14377
14378 export function validateCustomRange(
14379   start: string,
14380   end: string
14381 ): { valid: boolean; error?: string; startDate: string; endDate: string } {
14382   let startDate = start
14383   let endDate = end
14384
14385   const startMatch = /^\d{4}-\d{2}-\d{2}$/.test(startDate)
14386   const endMatch = /^\d{4}-\d{2}-\d{2}$/.test(endDate)
14387
14388   if (!startMatch || !endMatch) {
14389     return { valid: false, error: '日期格式必须为 yyyy-MM-dd', startDate, endDate }
14390   }
14391
14392   let startParsed: Date
14393   let endParsed: Date
14394
14395   try {
14396     startParsed = parseISO(startDate)
14397     endParsed = parseISO(endDate)
14398   } catch {
14399     return { valid: false, error: '日期解析失败', startDate, endDate }
14400   }
14401
14402   if (isNaN(startParsed.getTime()) || isNaN(endParsed.getTime())) {
14403     return { valid: false, error: '无效的日期', startDate, endDate }
14404   }
14405
14406   if (startParsed > endParsed) {
14407     ;[startDate, endDate] = [endDate, startDate]
14408     ;[startParsed, endParsed] = [endParsed, startParsed]
14409   }
14410
```

```
  const diffDays = differenceInCalendarDays(endParsed, startParsed)
  if (diffDays > 365) {
    return { valid: false, error: '跨度不能超过 365 天', startDate, endDate }
  }

  return { valid: true, startDate, endDate }
}

export function filterLogsByRange(
  logs: DailyLog[],
  startDate: string,
  endDate: string
): DailyLog[] {
  return logs.filter((log) => log.date >= startDate && log.date <= endDate)
}

export function computeSummary(filteredLogs: DailyLog[]): SummaryResult {
  const subjectHoursMap = new Map<string, number>()
  const dateSet = new Set<string>()
  const dailyMap = new Map<string, { totalHours: number; subjects: Map<string, DailyLogSubject
> }>()

  let totalHours = 0

  for (const log of filteredLogs) {
    dateSet.add(log.date)

    let dailyEntry = dailyMap.get(log.date)
    if (!dailyEntry) {
      dailyEntry = { totalHours: 0, subjects: new Map() }
      dailyMap.set(log.date, dailyEntry)
    }

    for (const subj of log.subjects) {
      totalHours += subj.hours
      dailyEntry.totalHours += subj.hours

      const existing = subjectHoursMap.get(subj.id) ?? 0
      subjectHoursMap.set(subj.id, existing + subj.hours)

      const dailySubj = dailyEntry.subjects.get(subj.id)
      if (dailySubj) {
        dailySubj.hours += subj.hours
        if (subj.summary) {
          dailySubj.summary = dailySubj.summary ? dailySubj.summary + '; ' + subj.summary : su
bj.summary
        }
      } else {
        dailyEntry.subjects.set(subj.id, { ...subj })
      }
    }
  }

  const subjectBreakdown: SubjectBreakdown[] = []
  for (const [subjectId, hours] of subjectHoursMap) {
```

```
    const subject = getSubjectById(subjectId)
    const name = subject?.name ?? subjectId
    const percentage = totalHours > 0 ? (hours / totalHours) * 100 : 0
    subjectBreakdown.push({ subjectId, name, hours, percentage })
  }

  subjectBreakdown.sort((a, b) => b.hours - a.hours)

  if (subjectBreakdown.length > 0 && totalHours > 0) {
    const roundedPercentages = subjectBreakdown.map((s) => Math.round(s.percentage * 100) / 10
0)
    const sumRounded = roundedPercentages.reduce((a, b) => a + b, 0)
    const diff = Math.round((100 - sumRounded) * 100) / 100

    if (diff !== 0) {
      let maxIdx = 0
      for (let i = 1; i < subjectBreakdown.length; i++) {
        if (subjectBreakdown[i].hours > subjectBreakdown[maxIdx].hours) {
          maxIdx = i
        }
      }
      roundedPercentages[maxIdx] = Math.round((roundedPercentages[maxIdx] + diff) * 100) / 100
    }

    for (let i = 0; i < subjectBreakdown.length; i++) {
      subjectBreakdown[i].percentage = roundedPercentages[i]
    }
  }

  const sortedDates = Array.from(dailyMap.keys()).sort()
  const dailyTrend: DailyTrendItem[] = sortedDates.map((date) => {
    const entry = dailyMap.get(date)!
    return {
      date,
      totalHours: entry.totalHours,
      subjects: Array.from(entry.subjects.values()),
    }
  })

  return {
    totalHours,
    checkedDays: dateSet.size,
    subjectBreakdown,
    dailyTrend,
  }
}

// ==================== 文件: src/lib/supabase.ts ====================
import { createClient } from '@supabase/supabase-js'
import { Capacitor } from '@capacitor/core'
import { Preferences } from '@capacitor/preferences'

let supabaseUrl = (import.meta.env.VITE_SUPABASE_URL as string | undefined) ?? ''
const supabaseAnonKey = (import.meta.env.VITE_SUPABASE_ANON_KEY as string | undefined) ?? ''

```

```
/** 是否已正确配置 Supabase 环境变量（未配置时 UI 应展示友好错误而非白屏） */
export const isSupabaseConfigured = Boolean(supabaseUrl.trim() && supabaseAnonKey)

if (!isSupabaseConfigured) {
  console.error(
    '[Supabase] 缺少环境变量 VITE_SUPABASE_URL 或 VITE_SUPABASE_ANON_KEY，请检查部署环境配置。\n' +
    '部署平台（Vercel/CloudBase）需要在环境变量中设置这两个值，或在项目根目录提供 .env.production。'
  )
}

// 规范化 URL：
// 1. 去掉首尾空白
// 2. 确保有 https:// 前缀
// 3. 剥离多余的路径后缀（如 /rest/v1、/auth/v1、/storage/v1、/realtime/v1 等）
// 4. 去掉末尾斜杠
// 未配置环境变量时使用占位 URL，避免 createClient 抛错导致整站白屏（错误由 UI 层展示）
const PLACEHOLDER_URL = 'https://placeholder.supabase.co'
const PLACEHOLDER_KEY = 'placeholder-anon-key'

if (!isSupabaseConfigured) {
  supabaseUrl = PLACEHOLDER_URL
} else {
  try {
    const rawUrl = supabaseUrl.trim()
    const urlObj = new URL(rawUrl.startsWith('http') ? rawUrl : `https://${rawUrl}`)
    // 只保留 protocol + host，丢弃 pathname（用户常误填 /rest/v1 等路径）
    const normalized = `${urlObj.protocol}//${urlObj.hostname}`

    if (import.meta.env.DEV && urlObj.pathname && urlObj.pathname !== '/') {
      console.warn(
        `[Supabase] 自动修正 URL：原 URL 包含多余路径 "${urlObj.pathname}"，已剥离。\n` +
        `  原值: ${supabaseUrl}\n` +
        `  修正: ${normalized}\n` +
        `  建议将 VITE_SUPABASE_URL 改为仅包含域名（无路径）。`
      )
    }

    supabaseUrl = normalized
  } catch {
    // URL 解析失败时，兜底的简单规范化
    if (import.meta.env.DEV) console.warn('[Supabase] URL 解析失败，使用兜底规范化方式')
    supabaseUrl = supabaseUrl.trim().replace(/\/+$/, '')
    // 剥离常见的路径后缀
    supabaseUrl = supabaseUrl.replace(/\/(rest|auth|storage|realtime)\/v\d+.*$/i, '')
    if (!/^https?:\/\//i.test(supabaseUrl)) {
      supabaseUrl = `https://${supabaseUrl}`
    }
  }

  // 验证 Supabase URL 格式：必须是 https://<project-id>.supabase.co 格式
  const supabaseUrlPattern = /^https:\/\/[a-z0-9]+\.supabase\.co$/
  if (import.meta.env.DEV && !supabaseUrlPattern.test(supabaseUrl)) {
    console.warn(
```

```
      `[Supabase] URL 格式可能不正确: "${supabaseUrl}"\n` +
      '期望格式: https://<project-id>.supabase.co\n' +
      '请检查 VITE_SUPABASE_URL 是否正确。'
    )
  }
}

// 带超时的 fetch 封装，避免在移动端因网络问题长时间挂起
const REQUEST_TIMEOUT_MS = 10000 // 10 秒超时

function fetchWithTimeout(input: RequestInfo | URL, init?: RequestInit): Promise<Response> {
  const controller = new AbortController()
  const timeoutId = setTimeout(() => controller.abort(), REQUEST_TIMEOUT_MS)

  return fetch(input, {
    ...init,
    signal: controller.signal,
  }).finally(() => clearTimeout(timeoutId))
}

/**
 * Capacitor 原生环境下的 Storage Adapter
 * 将 Supabase session 持久化到原生 Preferences（SharedPreferences / NSUserDefaults），
 * 比 WebView 的 localStorage 更可靠，App 升级或系统清理缓存后不会丢失登录态。
 * Web 环境继续使用 Supabase 默认的 localStorage。
 */
const nativeStorageAdapter = {
  getItem: async (key: string) => {
    const { value } = await Preferences.get({ key })
    return value ?? null
  },
  setItem: async (key: string, value: string) => {
    await Preferences.set({ key, value })
  },
  removeItem: async (key: string) => {
    await Preferences.remove({ key })
  },
}

export const supabase = createClient(
  isSupabaseConfigured ? supabaseUrl : PLACEHOLDER_URL,
  isSupabaseConfigured ? supabaseAnonKey : PLACEHOLDER_KEY,
  {
    global: {
      fetch: fetchWithTimeout,
    },
    auth: {
      // 原生环境用 Preferences 存储 session；Web 环境用默认 localStorage
      storage: Capacitor.isNativePlatform() ? nativeStorageAdapter : undefined,
      persistSession: true,
      autoRefreshToken: true,
    },
  }
)

```

```
/** 最新 APK 下载地址（Supabase Storage 公开桶，作为网站/App 的 APK 下载源） */
export const apkDownloadUrl = supabase.storage
  .from('ota-bundles')
  .getPublicUrl('DiveDeep.apk').data.publicUrl

// ==================== 文件: src/lib/theme.ts ====================
const THEME_STORAGE_KEY = 'app_theme'

/** 主题：light/dark 为纯色基础主题；dawn（晨光）/dusk（暮色）/starry（午夜繁星）为氛围主题 */
export type ThemeMode = 'light' | 'dark' | 'dawn' | 'dusk' | 'starry'

export interface ThemeMeta {
  key: ThemeMode
  name: string
  desc: string
  /** 是否属于暗色系（决定 html.dark 是否启用） */
  dark: boolean
  /** 主题选择器里的渐变色块 */
  swatch: string
}

export const THEMES: ThemeMeta[] = [
  { key: 'light', name: '浅色', desc: '纯净白', dark: false, swatch: 'linear-gradient(135deg,#f8fafc,#e2e8f0)' },
  { key: 'dark', name: '深色', desc: '深夜蓝', dark: true, swatch: 'linear-gradient(135deg,#1e293b,#0f172a)' },
  { key: 'dawn', name: '晨光', desc: '晨曦渐变', dark: false, swatch: 'linear-gradient(135deg,#fdf4ff,#c7d2fe,#a5f3fc)' },
  { key: 'dusk', name: '暮色', desc: '星空渐变', dark: true, swatch: 'linear-gradient(135deg,#312e81,#0f172a,#1e3a8a)' },
  { key: 'starry', name: '午夜繁星', desc: '星际闪烁', dark: true, swatch: 'radial-gradient(circle at 30% 25%, #f8fafc 0 1px, transparent 2px), radial-gradient(circle at 65% 45%, #bae6fd 0 1px, transparent 2px), linear-gradient(135deg,#11173a,#05060f)' },
]

const DARK_THEMES = new Set<ThemeMode>(['dark', 'dusk', 'starry'])
const VALID: ThemeMode[] = ['light', 'dark', 'dawn', 'dusk', 'starry']

function isTheme(v: unknown): v is ThemeMode {
  return typeof v === 'string' && (VALID as string[]).includes(v)
}

/** 读取持久化主题；无记录时跟随系统偏好 */
export function getInitialTheme(): ThemeMode {
  try {
    const saved = localStorage.getItem(THEME_STORAGE_KEY)
    if (isTheme(saved)) return saved
  } catch {
    /* ignore */
  }
  if (typeof window !== 'undefined' && window.matchMedia) {
    return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'
  }
  return 'light'
}
```

```
14675
14676 /** 当前生效的主题（供组件初始化显示） */
14677 export function getCurrentTheme(): ThemeMode {
14678   const attr = document.documentElement.getAttribute('data-theme')
14679   if (isTheme(attr)) return attr
14680   return document.documentElement.classList.contains('dark') ? 'dark' : 'light'
14681 }
14682
14683 /** 应用主题到 <html>：data-theme 标识主题，dark class 决定 dark: 变体是否生效 */
14684 export function applyTheme(mode: ThemeMode) {
14685   const root = document.documentElement
14686   root.classList.toggle('dark', DARK_THEMES.has(mode))
14687   root.setAttribute('data-theme', mode)
14688 }
14689
14690 /** 设置并持久化主题 */
14691 export function setTheme(mode: ThemeMode): ThemeMode {
14692   try {
14693     localStorage.setItem(THEME_STORAGE_KEY, mode)
14694   } catch {
14695     /* ignore */
14696   }
14697   applyTheme(mode)
14698   return mode
14699 }
14700
14701 /** 依次轮换主题：light → dark → dawn → dusk → light */
14702 export function nextTheme(): ThemeMode {
14703   const current = getCurrentTheme()
14704   const idx = VALID.indexOf(current)
14705   const next = VALID[(idx + 1) % VALID.length]
14706   return setTheme(next)
14707 }
14708
14709 // ==================== 文件: src/lib/timerSync.ts ====================
14710 import type { DailyLogSubject } from './dailyLogs'
14711 import { sortSubjectsByStartTime, todayStr, upsertLogSafely } from './dailyLogs'
14712
14713 /**
14714  * 全功能模式计时器（StudyTimer）与精简挂件计时器（DesktopTimer）共享的
14715  * 「进行中」状态。两套 UI 读写同一个 key，保证在两种模式间切换时计时不中断：
14716  * - 全功能里开始计时 → 切到精简挂件仍显示计时，可直接结束/打卡
14717  * - 精简挂件里开始计时 → 进入全功能计时页同样恢复
14718  */
14719
14720 export interface SharedTimerState {
14721   subjectId: string | null
14722   activity: string
14723   startTime: number // Date.now()
14724   paused?: boolean // 是否暂停中（硬件 OK 键）
14725   pausedMs?: number // 累计已暂停的毫秒数
14726   pausedAt?: number | null // 本次暂停的开始时间戳
14727 }
14728
14729 /** 统一的进行中计时 key */
```

```
14730 export const TIMER_RUNNING_KEY = 'kaoyan_timer_running'
14731 /** 旧版精简挂件专用 key（仅迁移用，之后统一用 TIMER_RUNNING_KEY） */
14732 const LEGACY_WIDGET_KEY = 'kaoyan_widget_running'
14733
14734 function clearAllRunning() {
14735   localStorage.removeItem(TIMER_RUNNING_KEY)
14736   localStorage.removeItem(LEGACY_WIDGET_KEY)
14737 }
14738
14739 /** 读取共享的进行中计时（兼容旧版挂件格式，自动迁移）；跨午夜丢弃 */
14740 export function loadSharedTimer(): SharedTimerState | null {
14741   try {
14742     const raw = localStorage.getItem(TIMER_RUNNING_KEY) ?? localStorage.getItem(LEGACY_WIDGET_
      KEY)
14743     if (!raw) return null
14744     const parsed = JSON.parse(raw) as Partial<SharedTimerState>
14745     if (!parsed.startTime) {
14746       clearAllRunning()
14747       return null
14748     }
14749     // 跨午夜修复：若计时开始于今天之前，丢弃（避免把昨天甚至更早的时长计入今日）
14750     if (new Date(parsed.startTime).toDateString() !== new Date().toDateString()) {
14751       clearAllRunning()
14752       return null
14753     }
14754     const state: SharedTimerState = {
14755       subjectId: parsed.subjectId ?? null,
14756       activity: parsed.activity ?? '',
14757       startTime: parsed.startTime,
14758       paused: parsed.paused ?? false,
14759       pausedMs: parsed.pausedMs ?? 0,
14760       pausedAt: parsed.pausedAt ?? null,
14761     }
14762     // 若命中旧版挂件 key，迁移到统一 key 并清理旧值
14763     if (!localStorage.getItem(TIMER_RUNNING_KEY)) {
14764       localStorage.setItem(TIMER_RUNNING_KEY, JSON.stringify(state))
14765     }
14766     localStorage.removeItem(LEGACY_WIDGET_KEY)
14767     return state
14768   } catch {
14769     return null
14770   }
14771 }
14772
14773 export function saveSharedTimer(state: SharedTimerState | null): void {
14774   if (state) {
14775     localStorage.setItem(TIMER_RUNNING_KEY, JSON.stringify(state))
14776   } else {
14777     localStorage.removeItem(TIMER_RUNNING_KEY)
14778   }
14779 }
14780
14781 /** 已流逝秒数（暂停期间不计时） */
14782 export function computeTimerElapsed(state: SharedTimerState): number {
14783   const pausedMs = state.pausedMs ?? 0
```

```
14784   const now = state.paused ? (state.pausedAt ?? Date.now()) : Date.now()
14785   return Math.floor((now - state.startTime - pausedMs) / 1000)
14786 }
14787
14788 /** 暂停共享计时：冻结时长（恢复后继续累计）。未在计时或已暂停时忽略。 */
14789 export function pauseSharedTimer(): void {
14790   const running = loadSharedTimer()
14791   if (!running || running.paused) return
14792   saveSharedTimer({ ...running, paused: true, pausedAt: Date.now() })
14793 }
14794
14795 /** 恢复共享计时：把本次暂停的时长累计进 pausedMs。未暂停时忽略。 */
14796 export function resumeSharedTimer(): void {
14797   const running = loadSharedTimer()
14798   if (!running || !running.paused) return
14799   const add = Date.now() - (running.pausedAt ?? Date.now())
14800   saveSharedTimer({
14801     ...running,
14802     paused: false,
14803     pausedAt: null,
14804     pausedMs: (running.pausedMs ?? 0) + add,
14805   })
14806 }
14807
14808 /** 生成一条待入库的科目条目（含时间段） */
14809 export function buildTimerEntry(state: SharedTimerState, endSeconds: number): DailyLogSubject
      | null {
14810   if (!state.subjectId) return null
14811   const hours = Math.round((endSeconds / 3600) * 100) / 100
14812   const endNow = state.paused ? (state.pausedAt ?? Date.now()) : Date.now()
14813   return {
14814     id: state.subjectId,
14815     hours,
14816     ...(state.activity ? { activity: state.activity } : {}),
14817     startTime: timeStr(new Date(state.startTime)),
14818     endTime: timeStr(new Date(endNow)),
14819   }
14820 }
14821
14822 /* ── 本地待补记备份 ──
14823    精简挂件结束计时时先把本次会话落一份到 localStorage，云端保存成功后才清除。
14824    登录/网络/会话异常导致保存失败时，时长不会丢：全功能「计时」页挂载时自动恢复为今日累计，
14825    再由用户手动保存（或补交）。 */
14826 export interface PendingTimerEntry {
14827   subjectId: string
14828   activity: string
14829   seconds: number
14830   start: string // HH:mm
14831   end: string // HH:mm
14832   date: string // yyyy-MM-dd（计时开始那天，跨天补交依据）
14833 }
14834
14835 const PENDING_KEY = 'kaoyan_pending_timer'
14836
14837 export function savePendingTimer(entry: PendingTimerEntry): void {
```

```
14838   try {
14839     localStorage.setItem(PENDING_KEY, JSON.stringify(entry))
14840   } catch {
14841     /* ignore */
14842   }
14843 }
14844
14845 export function loadPendingTimer(): PendingTimerEntry | null {
14846   try {
14847     const raw = localStorage.getItem(PENDING_KEY)
14848     if (!raw) return null
14849     const p = JSON.parse(raw) as Partial<PendingTimerEntry>
14850     if (typeof p.subjectId !== 'string' || typeof p.seconds !== 'number' || typeof p.date !==
      'string') return null
14851     return {
14852       subjectId: p.subjectId,
14853       activity: p.activity ?? '',
14854       seconds: p.seconds,
14855       start: p.start ?? '',
14856       end: p.end ?? '',
14857       date: p.date,
14858     }
14859   } catch {
14860     return null
14861   }
14862 }
14863
14864 export function clearPendingTimer(): void {
14865   try {
14866     localStorage.removeItem(PENDING_KEY)
14867   } catch {
14868     /* ignore */
14869   }
14870 }
14871
14872 /** 时间戳 → HH:mm（本地时区） */
14873 export function timeHm(ts: number): string {
14874   const d = new Date(ts)
14875   const pad = (n: number) => String(n).padStart(2, '0')
14876   return `${pad(d.getHours())}:${pad(d.getMinutes())}`
14877 }
14878
14879 /** 时间戳 → yyyy-MM-dd（本地时区） */
14880 export function dateOf(ts: number): string {
14881   const d = new Date(ts)
14882   const pad = (n: number) => String(n).padStart(2, '0')
14883   return `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}`
14884 }
14885
14886 function timeStr(d: Date): string {
14887   const pad = (n: number) => String(n).padStart(2, '0')
14888   return `${pad(d.getHours())}:${pad(d.getMinutes())}`
14889 }
14890
14891 /* ── 结束共享计时并打卡 ──
```

```
14892    胶囊条与精简面板共用的结束逻辑：先停表并本地备份本次会话，
14893    再按登录状态与是否跨天决定直接入库或提示补交。 */
14894
14895 export type FinishTimerResult =
14896   | { status: 'saved'; seconds: number }
14897   | { status: 'stopped'; message: string }
14898
14899 /**
14900  * 结束共享计时并尝试为用户记录打卡。返回：
14901  * - saved：已成功写入今日记录（seconds 为实际秒数）
14902  * - stopped：已停表但未入库，message 是需要 UI 展示的提示（本地暂存/跨天补交/未登录/保存失败
      ）
14903  */
14904 export async function finishSharedTimer(user: { id: string } | null): Promise<FinishTimerResul
      t> {
14905   const running = loadSharedTimer()
14906   if (!running) return { status: 'stopped', message: '' }
14907   const seconds = computeTimerElapsed(running)
14908   const entry = buildTimerEntry(running, seconds)
14909   // 无论是否入库成功，计时都先停止（清掉共享计时，避免两个界面重复操作）
14910   saveSharedTimer(null)
14911   if (!entry || seconds < 1) return { status: 'stopped', message: '' }
14912
14913   // 云端保存前先落一份本地备份：登录/网络/会话异常导致保存失败时本次时长不丢，
14914   // 之后在全功能「计时」页挂载时自动恢复为「今日累计」并可再次保存/补交
14915   savePendingTimer({
14916     subjectId: running.subjectId ?? '',
14917     activity: running.activity ?? '',
14918     seconds,
14919     start: timeHm(running.startTime),
14920     end: timeHm(running.paused ? (running.pausedAt ?? Date.now()) : Date.now()),
14921     date: dateOf(running.startTime),
14922   })
14923
14924   if (!user) {
14925     return { status: 'stopped', message: '已暂存本地，登录后在全功能「计时」页保存即可' }
14926   }
14927   const targetDate = dateOf(running.startTime)
14928   if (targetDate !== todayStr()) {
14929     // 跨天补交：交给全功能计时页的补交流程（带确认），这里只保全不写错日期
14930     return { status: 'stopped', message: '已暂存本地（跨天补交），请到全功能「计时」页保存' }
14931   }
14932   try {
14933     const entries: DailyLogSubject[] = [entry]
14934     sortSubjectsByStartTime(entries)
14935     // 安全合并写入云端：内部自动读最新→合并→带版本校验写入，并发冲突自动重试一次
14936     await upsertLogSafely({ userId: user.id, date: targetDate, subjects: entries, summary: ''
      })
14937     clearPendingTimer() // 云端写入成功才清理本地备份
14938     return { status: 'saved', seconds }
14939   } catch (err) {
14940     return {
14941       status: 'stopped',
14942       message:
14943         '保存失败，时长已暂存本地，可到全功能「计时」页重试：' +
```

```
        (err instanceof Error ? err.message : '未知错误'),
    }
  }
}

// ==================== 文件: src/lib/Toast.tsx ====================
// 轻量 Toast 提示
import { createContext, useContext, useCallback, useRef, useState, type ReactNode } from 'react'

interface Toast {
  id: number
  message: string
  icon?: string
  duration: number
}

interface ToastContextValue {
  show: (message: string, opts?: { icon?: string; duration?: number }) => void
}

const ToastContext = createContext<ToastContextValue | null>(null)

export function useToast(): ToastContextValue {
  const ctx = useContext(ToastContext)
  if (!ctx) throw new Error('useToast must be used within ToastProvider')
  return ctx
}

let toastIdCounter = 0

export function ToastProvider({ children }: { children: ReactNode }) {
  const [toasts, setToasts] = useState<Toast[]>([])
  const timersRef = useRef<Map<number, ReturnType<typeof setTimeout>>>(new Map())

  const dismiss = useCallback((id: number) => {
    setToasts((prev) => prev.filter((t) => t.id !== id))
    const timer = timersRef.current.get(id)
    if (timer) {
      clearTimeout(timer)
      timersRef.current.delete(id)
    }
  }, [])

  // 只在 toasts 变化时清理空 timer 不做额外逻辑
  void toasts

  const show = useCallback(
    (message: string, opts?: { icon?: string; duration?: number }) => {
      const id = ++toastIdCounter
      const duration = opts?.duration ?? 3000
      setToasts((prev) => [...prev, { id, message, icon: opts?.icon, duration }])
      const timer = setTimeout(() => dismiss(id), duration)
      timersRef.current.set(id, timer)
    },
```

```
14998     [dismiss],
14999   )
15000
15001   return (
15002     <ToastContext.Provider value={{ show }}>
15003       {children}
15004       {/* Toast 容器 */}
15005       <div className="fixed top-16 inset-x-0 z-[100] flex flex-col items-center gap-2 px-4 poi
      nter-events-none">
15006         {toasts.map((t) => (
15007           <button
15008             key={t.id}
15009             type="button"
15010             onClick={() => dismiss(t.id)}
15011             className="pointer-events-auto max-w-sm w-auto flex items-center gap-2 rounded-xl
      bg-gray-900/95 dark:bg-slate-800/95 text-white px-4 py-2.5 text-sm shadow-lg border border-white/10
      animate-[toastIn_.25s_ease-out]"
15012           >
15013             {t.icon && <span className="shrink-0 text-base leading-none">{t.icon}</span>}
15014             <span className="leading-snug">{t.message}</span>
15015           </button>
15016         ))}
15017       </div>
15018       <style>{`
15019         @keyframes toastIn {
15020           from { opacity: 0; transform: translateY(-8px); }
15021           to { opacity: 1; transform: translateY(0); }
15022         }
15023       `}</style>
15024     </ToastContext.Provider>
15025   )
15026 }
15027
15028 // ==================== 文件: src/lib/vocabulary.ts ====================
15029 // 生词本工具：管理标记单词的持久化存储和生词本数据（支持云端同步）
15030 import { supabase } from './supabase'
15031 import { isSupabaseConfigured } from './supabase'
15032
15033 const MARKED_WORDS_KEY = 'english_marked_words'
15034 const VOCABULARY_KEY = 'english_vocabulary'
15035
15036 export interface VocabWord {
15037   word: string
15038   sentence: string
15039   day: number
15040   sentIdx: number
15041   wordIdx: number
15042   addedAt: string // ISO date string
15043 }
15044
15045 export interface VocabDay {
15046   day: number
15047   words: VocabWord[]
15048 }
15049
```

```
/** 单词在本地的唯一标识（用于去重），与云端列对应 */
function wordKey(w: Pick<VocabWord, 'word' | 'sentence'>): string {
  return `${w.word.toLowerCase()}|${w.sentence}`
}

// ---------- 标记单词（红色标记）持久化（纯本地） ----------

/** 从 localStorage 读取已标记的单词 key 集合 */
export function loadMarkedWords(): Set<string> {
  try {
    const raw = localStorage.getItem(MARKED_WORDS_KEY)
    if (!raw) return new Set()
    const arr: string[] = JSON.parse(raw)
    return new Set(arr)
  } catch {
    return new Set()
  }
}

/** 将已标记的单词 key 集合保存到 localStorage */
export function saveMarkedWords(words: Set<string>): void {
  localStorage.setItem(MARKED_WORDS_KEY, JSON.stringify([...words]))
}

// ---------- 生词本持久化（本地缓存） ----------

/** 从 localStorage 读取本地生词本缓存 */
export function loadVocabulary(): VocabDay[] {
  try {
    const raw = localStorage.getItem(VOCABULARY_KEY)
    if (!raw) return []
    return JSON.parse(raw) as VocabDay[]
  } catch {
    return []
  }
}

/** 保存生词本到本地缓存 */
export function saveVocabulary(data: VocabDay[]): void {
  localStorage.setItem(VOCABULARY_KEY, JSON.stringify(data))
}

/** 将 VocabDay[] 展平为单词数组 */
function flattenDays(vocab: VocabDay[]): VocabWord[] {
  return vocab.flatMap((d) => d.words)
}

/** 按 day 分组还原为 VocabDay[]（保持 day 升序、每组内 words 保持原序） */
function groupByDay(words: VocabWord[]): VocabDay[] {
  const map = new Map<number, VocabWord[]>()
  for (const w of words) {
    if (!map.has(w.day)) map.set(w.day, [])
    map.get(w.day)!.push(w)
  }
  return [...map.entries()]
```

```
15105     .sort((a, b) => a[0] - b[0])
15106     .map(([day, list]) => ({ day, words: list }))
15107 }
15108
15109 // ---------- 云端同步接口 ----------
15110
15111 interface CloudVocabRow {
15112   day: number
15113   sentIdx: number
15114   wordIdx: number
15115   word: string
15116   sentence: string
15117   added_at: string
15118 }
15119
15120 function toWord(r: CloudVocabRow): VocabWord {
15121   return {
15122     word: r.word,
15123     sentence: r.sentence,
15124     day: r.day,
15125     sentIdx: r.sentIdx,
15126     wordIdx: r.wordIdx,
15127     addedAt: r.added_at || new Date().toISOString(),
15128   }
15129 }
15130
15131 /**
15132  * 拉取云端生词，与本地缓存合并（并集去重），回写本地。
15133  * 合并规则：同一 day+sentIdx+wordIdx 视为同一位置，云端与本地冲突时保留 addedAt 更早的一条（
      操作稳定）。
15134  * 返回合并后的结果。
15135  */
15136 export async function syncVocabularyFromCloud(userId: string): Promise<VocabDay[]> {
15137   if (!isSupabaseConfigured) return loadVocabulary()
15138
15139   const local = flattenDays(loadVocabulary())
15140   let cloud: VocabWord[] = []
15141   try {
15142     const { data, error } = await supabase
15143       .from('user_vocab')
15144       .select('day, sentIdx, wordIdx, word, sentence, added_at')
15145       .eq('user_id', userId)
15146     if (error) throw new Error(error.message)
15147     cloud = ((data as unknown) as CloudVocabRow[]).map(toWord)
15148   } catch (err) {
15149     // 云端读取失败（如表未建/未授权）：保留本地数据，静默降级
15150     console.warn('[Vocab] cloud sync read failed, keep local:', err instanceof Error ? err.mes
      sage : err)
15151     return loadVocabulary()
15152   }
15153
15154   // 合并：以 位置键(day-sentIdx-wordIdx) 为主键
15155   const merged = new Map<string, VocabWord>()
15156   const localByPos = new Map<string, VocabWord>()
15157   for (const w of local) localByPos.set(`${w.day}|${w.sentIdx}|${w.wordIdx}`, w)
```

```
15158   for (const w of cloud) merged.set(`${w.day}|${w.sentIdx}|${w.wordIdx}`, w)
15159
15160   // 本地有而云端没有的位置，补回云端（本地兜底上传）
15161   for (const [pos, w] of localByPos) {
15162     if (!merged.has(pos)) merged.set(pos, w)
15163   }
15164   // 云端有而本地无的，补回本地
15165   const mergedWords = [...merged.values()]
15166
15167   // 冲突时保留 earlier addedAt；若同一位置一条来自云端一条来自本地，以云端为准（它可能更新）
15168   // 但 merged 已按云端优先。这里仅做稳定排序
15169   mergedWords.sort((a, b) => {
15170     if (a.day !== b.day) return a.day - b.day
15171     if (a.sentIdx !== b.sentIdx) return a.sentIdx - b.sentIdx
15172     return a.wordIdx - b.wordIdx
15173   })
15174
15175   const result = groupByDay(mergedWords)
15176   saveVocabulary(result)
15177   return result
15178 }
15179
15180 /**
15181  * 将两个位置集合推送到云端（增量 upsert，按 位置唯一键）。
15182  * positions: { day, sentIdx, wordIdx, word, sentence }
15183  */
15184 export async function pushVocabularyToCloud(
15185   userId: string,
15186   words: Pick<VocabWord, 'day' | 'sentIdx' | 'wordIdx' | 'word' | 'sentence'>[],
15187 ): Promise<void> {
15188   if (!isSupabaseConfigured || words.length === 0) return
15189   const rows = words.map((w) => ({
15190     user_id: userId,
15191     day: w.day,
15192     sentIdx: w.sentIdx,
15193     wordIdx: w.wordIdx,
15194     word: w.word,
15195     sentence: w.sentence,
15196   }))
15197   try {
15198     await supabase.from('user_vocab').upsert(rows, { onConflict: 'user_id,day,sentIdx,wordIdx'
       })
15199   } catch (err) {
15200     console.warn('[Vocab] cloud push failed:', err instanceof Error ? err.message : err)
15201   }
15202 }
15203
15204 /**
15205  * 删除某单词在云端的记录（按 user+day+word+sentence 定位）。
15206  * 返回是否成功。
15207  */
15208 export async function removeWordFromCloud(
15209   userId: string,
15210   day: number,
15211   word: string,
```

```
  sentence: string,
): Promise<void> {
  if (!isSupabaseConfigured) return
  try {
    await supabase
      .from('user_vocab')
      .delete()
      .eq('user_id', userId)
      .eq('day', day)
      .eq('word', word)
      .eq('sentence', sentence)
  } catch (err) {
    console.warn('[Vocab] cloud remove failed:', err instanceof Error ? err.message : err)
  }
}

/** 清空云端生词本 */
export async function clearVocabularyFromCloud(userId: string): Promise<void> {
  if (!isSupabaseConfigured) return
  try {
    await supabase.from('user_vocab').delete().eq('user_id', userId)
  } catch (err) {
    console.warn('[Vocab] cloud clear failed:', err instanceof Error ? err.message : err)
  }
}

// ---------- 本地写操作（保持原有接口，供页面调用） ----------

/** 将某一天的标记单词存入生词本（合并去重，并返回本地更新后的结果） */
export function addDayToVocabulary(
  day: number,
  words: { word: string; sentence: string; sentIdx: number; wordIdx: number }[],
): VocabDay[] {
  const vocab = loadVocabulary()
  const existing = vocab.find((v) => v.day === day)

  const newWords: VocabWord[] = words.map((w) => ({
    ...w,
    day,
    addedAt: new Date().toISOString(),
  }))

  if (existing) {
    const existingKeys = new Set(existing.words.map((w) => wordKey(w)))
    const toAdd = newWords.filter((w) => !existingKeys.has(wordKey(w)))
    existing.words = [...existing.words, ...toAdd]
  } else {
    vocab.push({ day, words: newWords })
  }

  saveVocabulary(vocab)
  return vocab
}

/** 从生词本中删除某个单词（本地） */
```

```
15267 export function removeWordFromVocabulary(day: number, word: string, sentence: string): VocabDa
      y[] {
15268   const vocab = loadVocabulary()
15269   const dayEntry = vocab.find((v) => v.day === day)
15270   if (dayEntry) {
15271     dayEntry.words = dayEntry.words.filter((w) => !(w.word === word && w.sentence === sentence
      ))
15272     if (dayEntry.words.length === 0) {
15273       const idx = vocab.indexOf(dayEntry)
15274       vocab.splice(idx, 1)
15275     }
15276   }
15277   saveVocabulary(vocab)
15278   return vocab
15279 }
15280
15281 /** 清空本地生词本 */
15282 export function clearVocabulary(): void {
15283   localStorage.removeItem(VOCABULARY_KEY)
15284 }
15285
15286 // ==================== 文件: src/lib/wordLookup.ts ====================
15287 // 生词本离线查词：优先本地离线考研词典（0 费用），词库未收录才回退 AI（腾讯云 SCF / Supabase
      Edge Function）
15288 import { loadOfflineDict } from '../data/offlineDict'
15289 import { supabase } from './supabase'
15290 import { postJson } from './httpRequest'
15291
15292 export interface WordLookup {
15293   word: string
15294   phonetic: string
15295   meanings: string[]
15296   mnemonic: string
15297   collocations: string[]
15298   example: string
15299   examNote: string
15300 }
15301
15302 // 优先走腾讯云 SCF 云函数（国内节点直连讯飞，稳定快速）；未配置时回退到 Supabase Edge Functio
      n
15303 const AI_CORRECT_URL = (import.meta.env.VITE_AI_CORRECT_URL as string | undefined)?.trim() ||
      ''
15304
15305 const LOOKUP_CACHE_KEY = 'english_word_lookup_cache'
15306
15307 // ---------- 本地缓存（按单词缓存，重复查看不消耗 token） ----------
15308
15309 export function loadLookupCache(): Record<string, WordLookup> {
15310   try {
15311     const raw = localStorage.getItem(LOOKUP_CACHE_KEY)
15312     if (!raw) return {}
15313     return JSON.parse(raw) as Record<string, WordLookup>
15314   } catch {
15315     return {}
15316   }
```

```
15317 }
15318
15319 function saveLookupCache(cache: Record<string, WordLookup>): void {
15320   try {
15321     localStorage.setItem(LOOKUP_CACHE_KEY, JSON.stringify(cache))
15322   } catch {
15323     // 缓存写入失败时静默忽略，不影响查词
15324   }
15325 }
15326
15327 /** 读取某个单词的缓存结果（无则返回 null） */
15328 export function getCachedLookup(word: string): WordLookup | null {
15329   const key = word.toLowerCase()
15330   return loadLookupCache()[key] || null
15331 }
15332
15333 // ---------- 本地离线词典查询（0 费用，优先于 AI） ----------
15334
15335 /** 简单词形还原，匹配离线词库中的原形词（不足 3 字母直接返回原样） */
15336 function stemWord(w: string, dict: Record<string, string>): string {
15337   if (w.length <= 3) return w
15338   if (w.endsWith('ing') && w.length > 5) return w.slice(0, -3)
15339   if (w.endsWith('ied') && w.length > 4) return w.slice(0, -3) + 'y'
15340   if (w.endsWith('ed') && w.length > 4 && !w.endsWith('eed')) return w.slice(0, -2)
15341   if (w.endsWith('ies') && w.length > 4) return w.slice(0, -3) + 'y'
15342   if (w.endsWith('es') && w.length > 4 && dict[w.slice(0, -2)]) return w.slice(0, -2)
15343   if (w.endsWith('s') && w.length > 3 && dict[w.slice(0, -1)] && !w.endsWith('ss') && !w.endsW
      ith('us')) return w.slice(0, -1)
15344   return w
15345 }
15346
15347 /** 从离线词库查询释义（未收录返回 null）；词库为独立懒加载 chunk，首次调用时加载 */
15348 export async function lookupOffline(word: string): Promise<WordLookup | null> {
15349   const dict = await loadOfflineDict()
15350   const w = word.toLowerCase()
15351   const meaning = dict[w] || dict[stemWord(w, dict)]
15352   if (!meaning) return null
15353   return {
15354     word: w,
15355     phonetic: '',
15356     meanings: meaning.split(/[；;]/).map((s) => s.trim()).filter(Boolean),
15357     mnemonic: '',
15358     collocations: [],
15359     example: '',
15360     examNote: '',
15361   }
15362 }
15363
15364 // ---------- 调用后端查词（离线未收录时兜底） ----------
15365
15366 /** 查询单词的考研释义：缓存 → 本地离线词典 → AI 兜底 */
15367 export async function lookupWord(word: string): Promise<WordLookup> {
15368   const cached = getCachedLookup(word)
15369   if (cached) return cached
15370
```

```
15371   // 本地离线词典优先，0 费用；词典 chunk 加载失败时静默降级走 AI
15372   try {
15373     const offline = await lookupOffline(word)
15374     if (offline) return offline
15375   } catch { /* 词典加载失败，降级 AI */ }
15376
15377   let result: WordLookup
15378   if (AI_CORRECT_URL) {
15379     // 腾讯云 SCF 云函数；原生平台用原生网络栈，绕开 WebView fetch 崩溃黑屏
15380     const { ok, status, body } = await postJson(AI_CORRECT_URL, { action: 'lookup', word })
15381     if (!ok || !body?.ok) throw new Error(body?.error || `AI 查词失败（${status}）`)
15382     result = body.data
15383   } else {
15384     // 兜底：Supabase Edge Function
15385     const { data, error } = await supabase.functions.invoke('ai-correct', {
15386       body: { action: 'lookup', word },
15387     })
15388     if (error) throw new Error(error.message || 'AI 查词失败')
15389     result = data as WordLookup
15390   }
15391
15392   const cache = loadLookupCache()
15393   cache[word.toLowerCase()] = result
15394   saveLookupCache(cache)
15395   return result
15396 }
15397
15398 // ==================== 文件: src/data/englishDaily.ts ====================
15399 // 本文件类型定义 + 懒加载入口；数据体在 englishDaily.json（按需加载，独立 chunk）
15400 // 由 scripts/gen-english-daily.cjs / gen-ai-analysis.cjs 自动生成，请勿手改
15401
15402 export interface EnglishDaySentence {
15403   num: string
15404   en: string
15405   ref: string
15406   // 预生成的 AI 解析内容（句子主干/结构/搭配），由 scripts/gen-ai-analysis.cjs 批量生成，避免
      运行时消耗 AI token
15407   ai?: {
15408     backbone: string
15409     structure: string[]
15410     collocations: string[]
15411   }
15412 }
15413
15414 export interface VocabItem {
15415   raw: string
15416   word: string
15417   meaning: string
15418 }
15419
15420 export interface AnalysisItem {
15421   sentNum: string
15422   vocab: VocabItem[]
15423   split: string
15424   grammar: string[]
```

```
15425   ref: string
15426 }
15427
15428 export interface EnglishDay {
15429   day: number
15430   type: '英一' | '英二'
15431   source: string
15432   sentences: EnglishDaySentence[]
15433   zh: string
15434   analysis?: AnalysisItem[]
15435 }
15436
15437 // ---------- 懒加载（模块级缓存 + 并发去重） ----------
15438
15439 let cache: EnglishDay[] | null = null
15440 let pending: Promise<EnglishDay[]> | null = null
15441
15442 /** 动态 import JSON（独立异步 chunk，JSON.parse 解析比巨型 JS 字面量快数倍） */
15443 export function loadEnglishDaily(): Promise<EnglishDay[]> {
15444   if (cache) return Promise.resolve(cache)
15445   if (!pending) {
15446     pending = import('./englishDaily.json')
15447       .then((m) => {
15448         cache = m.default as EnglishDay[]
15449         return cache
15450       })
15451       .catch((e) => {
15452         pending = null // 失败后允许重试
15453         throw e
15454       })
15455   }
15456   return pending
15457 }
15458
15459 // ==================== 文件: src/data/offlineDict.ts ====================
15460 // 自动生成离线考研词典（合并 初中/高中/四级/六级/考研 词表）
15461 // 数据源：https://github.com/KyleBing/english-vocabulary (MIT)
15462 // 数据体在 offlineDict.json（按需加载，独立 chunk）
15463 // 仅供本地离线查词，勿手动编辑（重新生成：node scripts/build-offline-dict.cjs）
15464
15465 // ---------- 懒加载（模块级缓存 + 并发去重） ----------
15466
15467 let cache: Record<string, string> | null = null
15468 let pending: Promise<Record<string, string>> | null = null
15469
15470 /** 动态 import JSON（独立异步 chunk，只在真正查词时加载） */
15471 export function loadOfflineDict(): Promise<Record<string, string>> {
15472   if (cache) return Promise.resolve(cache)
15473   if (!pending) {
15474     pending = import('./offlineDict.json')
15475       .then((m) => {
15476         cache = m.default as Record<string, string>
15477         return cache
15478       })
15479       .catch((e) => {
```

```
        pending = null // 失败后允许重试
        throw e
      })
  }
  return pending
}

// ==================== 文件: src/plugins/timer-foreground/definitions.ts ====================
export type PermissionState = 'granted' | 'denied' | 'prompt' | 'prompt-with-rationale'

export interface BluetoothPermissionStatus {
  bluetooth: PermissionState
}

export interface TimerForegroundPlugin {
  startTimer(options: { subject: string; startTimeMs: number }): Promise<void>
  stopTimer(): Promise<void>
  updateTimer(options: { subject: string; elapsedSec: number }): Promise<void>
  addListener(eventName: 'timerStopped', listenerFunc: () => void): Promise<PluginListenerHandle>
  /** 请求 BLE 扫描/连接所需的运行时权限（Android 12+ 附近设备 / 旧版定位） */
  requestPermissions(): Promise<BluetoothPermissionStatus>
  /** 查询 BLE 相关权限状态 */
  checkPermissions(): Promise<BluetoothPermissionStatus>
}

export interface PluginListenerHandle {
  remove: () => Promise<void>
}

// ==================== 文件: src/plugins/timer-foreground/index.ts ====================
import { registerPlugin } from '@capacitor/core'
import type { TimerForegroundPlugin } from './definitions'
import { WebTimerForeground } from './web'

const TimerForeground = registerPlugin<TimerForegroundPlugin>('TimerForeground', {
  web: () => new WebTimerForeground(),
})

export { TimerForeground }
export type { TimerForegroundPlugin }

// ==================== 文件: src/plugins/timer-foreground/web.ts ====================
import type { PluginListenerHandle, TimerForegroundPlugin, BluetoothPermissionStatus } from './definitions'

export class WebTimerForeground implements TimerForegroundPlugin {
  async startTimer(): Promise<void> {
    // Web 端无需前台服务
  }
  async stopTimer(): Promise<void> {
    // Web 端无需前台服务
  }
  async updateTimer(): Promise<void> {
    // Web 端无需前台服务
```

```
15533   }
15534   async requestPermissions(): Promise<BluetoothPermissionStatus> {
15535     return { bluetooth: 'granted' }
15536   }
15537   async checkPermissions(): Promise<BluetoothPermissionStatus> {
15538     return { bluetooth: 'granted' }
15539   }
15540   async addListener(): Promise<PluginListenerHandle> {
15541     return { remove: async () => {} }
15542   }
15543 }
15544
15545 // ==================== 文件: src/index.css ====================
15546 @import "tailwindcss";
15547
15548 /* Tailwind v4：dark 变体基于 .dark class（手动切换主题生效），而非跟随系统 */
15549 @custom-variant dark (&:where(.dark, .dark *));
15550
15551 /* ── 设计令牌：字体 / 阴影 ── */
15552 @theme {
15553   /* 中文字体栈：优先系统字体，避免引入网络字体拖慢加载 */
15554   --font-sans: "HarmonyOS Sans SC", "PingFang SC", "Microsoft YaHei", "Noto Sans SC",
15555     ui-sans-serif, system-ui, -apple-system, "Segoe UI", Roboto, "Helvetica Neue", Arial,
15556     sans-serif;
15557
15558   /* 统一定义卡片/悬停/发光的阴影，页面不再各自手写 */
15559   --shadow-card: 0 1px 2px rgba(15, 23, 42, 0.04), 0 8px 24px -14px rgba(15, 23, 42, 0.1);
15560   --shadow-card-hover: 0 2px 4px rgba(15, 23, 42, 0.05), 0 12px 32px -16px rgba(79, 70, 229, 0
      .28);
15561   --shadow-glow: 0 4px 18px -6px rgba(79, 70, 229, 0.55);
15562 }
15563
15564 /* ── 全局基础 ── */
15565 html {
15566   -webkit-text-size-adjust: 100%;
15567 }
15568
15569 html.dark {
15570   color-scheme: dark;
15571 }
15572
15573 /* ── 扩展主题：渐变氛围背景 ──
15574    light/dark 是纯色底；dawn（晨光）/dusk（暮色）在主题根容器（.theme-surface）
15575    上叠一层柔和渐变，替换原来的纯色底，卡片等局部仍保持实色保证对比度 */
15576 html[data-theme='dawn'] body {
15577   background-color: #eef2ff;
15578 }
15579 html[data-theme='dusk'] body {
15580   background-color: #0f172a;
15581 }
15582 html[data-theme='dawn'] .theme-surface {
15583   background-color: transparent; /* 渐变接管纯色底 */
15584   background-image: linear-gradient(180deg, #fdf4ff 0%, #eef2ff 42%, #ecfeff 100%);
15585   background-attachment: fixed;
15586 }
```

```
15587 html[data-theme='dusk'] .theme-surface {
15588   background-color: transparent; /* 渐变接管纯色底 */
15589   background-image: linear-gradient(180deg, #221c4d 0%, #0f172a 55%, #172554 100%);
15590   background-attachment: fixed;
15591 }
15592
15593 /* ── 午夜繁星：深蓝夜空渐变 + 随机星星 + 错峰闪烁 ── */
15594 html[data-theme='starry'] .theme-surface {
15595   background-color: transparent; /* 星空接管纯色底 */
15596   background-image:
15597     /* 静态星群：固定不发光的细小星星 */
15598     radial-gradient(1.5px 1.5px at 4% 6%, rgba(255,255,255,0.85) 50%, transparent 51%),
15599     radial-gradient(1px 1px at 12% 22%, rgba(216,225,255,0.7) 50%, transparent 51%),
15600     radial-gradient(2px 2px at 18% 48%, rgba(255,255,255,0.8) 50%, transparent 51%),
15601     radial-gradient(1px 1px at 26% 72%, rgba(199,214,255,0.65) 50%, transparent 51%),
15602     radial-gradient(1.5px 1.5px at 33% 9%, rgba(255,255,255,0.75) 50%, transparent 51%),
15603     radial-gradient(1px 1px at 41% 33%, rgba(226,232,255,0.6) 50%, transparent 51%),
15604     radial-gradient(2px 2px at 49% 60%, rgba(255,255,255,0.9) 50%, transparent 51%),
15605     radial-gradient(1px 1px at 57% 84%, rgba(199,214,255,0.65) 50%, transparent 51%),
15606     radial-gradient(1.5px 1.5px at 64% 15%, rgba(255,255,255,0.75) 50%, transparent 51%),
15607     radial-gradient(1px 1px at 71% 40%, rgba(216,225,255,0.65) 50%, transparent 51%),
15608     radial-gradient(2px 2px at 79% 66%, rgba(255,255,255,0.8) 50%, transparent 51%),
15609     radial-gradient(1px 1px at 86% 90%, rgba(226,232,255,0.55) 50%, transparent 51%),
15610     radial-gradient(1.5px 1.5px at 92% 28%, rgba(255,255,255,0.75) 50%, transparent 51%),
15611     radial-gradient(1px 1px at 96% 55%, rgba(216,225,255,0.55) 50%, transparent 51%),
15612     /* 夜空渐变底色 */
15613     linear-gradient(180deg, #0d1134 0%, #090d26 48%, #04060f 100%);
15614 }
15615
15616 /* 两层错峰闪烁的星星（z-index:-1 保证星星在内容文字之下） */
15617 html[data-theme='starry'] .theme-surface::before,
15618 html[data-theme='starry'] .theme-surface::after {
15619   content: '';
15620   position: absolute;
15621   inset: 0;
15622   z-index: -1;
15623   border-radius: inherit;
15624   background-image:
15625     radial-gradient(1px 1px at 6% 30%, rgba(255,255,255,0.95) 50%, transparent 51%),
15626     radial-gradient(1.5px 1.5px at 15% 58%, rgba(226,232,255,0.8) 50%, transparent 51%),
15627     radial-gradient(1px 1px at 23% 13%, rgba(255,255,255,0.95) 50%, transparent 51%),
15628     radial-gradient(2px 2px at 30% 82%, rgba(255,255,255,0.85) 50%, transparent 51%),
15629     radial-gradient(1px 1px at 38% 44%, rgba(199,214,255,0.75) 50%, transparent 51%),
15630     radial-gradient(1.5px 1.5px at 46% 26%, rgba(255,255,255,0.9) 50%, transparent 51%),
15631     radial-gradient(1px 1px at 54% 70%, rgba(226,232,255,0.8) 50%, transparent 51%),
15632     radial-gradient(2px 2px at 62% 8%, rgba(255,255,255,0.95) 50%, transparent 51%),
15633     radial-gradient(1px 1px at 69% 50%, rgba(255,255,255,0.8) 50%, transparent 51%),
15634     radial-gradient(1.5px 1.5px at 77% 88%, rgba(216,225,255,0.85) 50%, transparent 51%),
15635     radial-gradient(1px 1px at 84% 36%, rgba(255,255,255,0.9) 50%, transparent 51%),
15636     radial-gradient(2px 2px at 91% 62%, rgba(226,232,255,0.8) 50%, transparent 51%),
15637     radial-gradient(1px 1px at 97% 18%, rgba(255,255,255,0.95) 50%, transparent 51%);
15638   pointer-events: none;
15639 }
15640
15641 /* 第二层星星换一批位置，与第一层错峰闪烁 */
```

```
15642 html[data-theme='starry'] .theme-surface::after {
15643   background-image:
15644     radial-gradient(1px 1px at 8% 55%, rgba(255,255,255,0.8) 50%, transparent 51%),
15645     radial-gradient(1.5px 1.5px at 17% 85%, rgba(226,232,255,0.7) 50%, transparent 51%),
15646     radial-gradient(1px 1px at 27% 38%, rgba(255,255,255,0.85) 50%, transparent 51%),
15647     radial-gradient(1px 1px at 36% 12%, rgba(199,214,255,0.7) 50%, transparent 51%),
15648     radial-gradient(2px 2px at 44% 68%, rgba(255,255,255,0.9) 50%, transparent 51%),
15649     radial-gradient(1px 1px at 53% 22%, rgba(226,232,255,0.75) 50%, transparent 51%),
15650     radial-gradient(1.5px 1.5px at 61% 90%, rgba(255,255,255,0.8) 50%, transparent 51%),
15651     radial-gradient(1px 1px at 68% 33%, rgba(199,214,255,0.7) 50%, transparent 51%),
15652     radial-gradient(1px 1px at 76% 72%, rgba(255,255,255,0.85) 50%, transparent 51%),
15653     radial-gradient(2px 2px at 83% 8%, rgba(226,232,255,0.8) 50%, transparent 51%),
15654     radial-gradient(1px 1px at 90% 48%, rgba(255,255,255,0.9) 50%, transparent 51%),
15655     radial-gradient(1.5px 1.5px at 95% 78%, rgba(216,225,255,0.75) 50%, transparent 51%);
15656   animation: star-twinkle-b 7.5s ease-in-out infinite;
15657 }
15658
15659 html[data-theme='starry'] .theme-surface::before {
15660   animation: star-twinkle-a 5.5s ease-in-out infinite;
15661 }
15662
15663 @keyframes star-twinkle-a {
15664   0%, 100% { opacity: 1; }
15665   50% { opacity: 0.3; }
15666 }
15667 @keyframes star-twinkle-b {
15668   0%, 100% { opacity: 0.3; }
15669   50% { opacity: 1; }
15670 }
15671
15672 /* 数字统一用等宽字形，统计类数据更整齐 */
15673 body {
15674   font-feature-settings: "tnum" 1;
15675   -webkit-font-smoothing: antialiased;
15676   -moz-osx-font-smoothing: grayscale;
15677   text-rendering: optimizeLegibility;
15678 }
15679
15680 html.dark ::selection {
15681   background: rgba(99, 102, 241, 0.35);
15682   color: #e2e8f0;
15683 }
15684
15685 ::selection {
15686   background: rgba(99, 102, 241, 0.22);
15687   color: #0f172a;
15688 }
15689
15690 /* 键盘可达性焦点提示（表单输入框用 focus:ring 自管，这里只管其余元素） */
15691 :focus-visible:not(input):not(textarea):not(select) {
15692   outline: 2px solid rgba(99, 102, 241, 0.55);
15693   outline-offset: 2px;
15694   border-radius: 6px;
15695 }
15696
```

```
15697 /* 滚动条 - 亮色 */
15698 ::-webkit-scrollbar {
15699   width: 10px;
15700   height: 10px;
15701 }
15702 ::-webkit-scrollbar-track {
15703   background: transparent;
15704 }
15705 ::-webkit-scrollbar-thumb {
15706   background: #cbd5e1;
15707   border-radius: 8px;
15708   border: 2px solid transparent;
15709   background-clip: content-box;
15710 }
15711 ::-webkit-scrollbar-thumb:hover {
15712   background: #94a3b8;
15713   background-clip: content-box;
15714   border: 2px solid transparent;
15715 }
15716
15717 /* 滚动条 - 暗色 */
15718 html.dark ::-webkit-scrollbar-thumb {
15719   background: #475569;
15720   background-clip: content-box;
15721   border: 2px solid transparent;
15722 }
15723 html.dark ::-webkit-scrollbar-thumb:hover {
15724   background: #64748b;
15725   background-clip: content-box;
15726   border: 2px solid transparent;
15727 }
15728
15729 /* ── 统一组件样式：全站卡片 / 按钮 / 输入框 ── */
15730 @layer components {
15731   /* 标准卡片：白卡 + 细边框 + 柔和阴影 */
15732   .card {
15733     @apply rounded-2xl border border-gray-200/70 bg-white shadow-[0_1px_2px_rgba(15,23,42,0.04
      ),0_8px_24px_-14px_rgba(15,23,42,0.1)] dark:border-slate-800 dark:bg-slate-900 dark:shadow-none;
15734   }
15735
15736   /* 可交互卡片：悬停轻微浮起 + 品牌色描边 */
15737   .card-hover {
15738     @apply rounded-2xl border border-gray-200/70 bg-white shadow-[0_1px_2px_rgba(15,23,42,0.04
      ),0_8px_24px_-14px_rgba(15,23,42,0.1)] transition-all duration-150 hover:-translate-y-0.5 hover:bord
      er-indigo-200 hover:shadow-[0_2px_4px_rgba(15,23,42,0.05),0_12px_32px_-16px_rgba(79,70,229,0.28)] da
      rk:border-slate-800 dark:bg-slate-900 dark:shadow-none dark:hover:border-indigo-500/40;
15739   }
15740
15741   /* 主按钮 */
15742   .btn-primary {
15743     @apply inline-flex cursor-pointer items-center justify-center gap-1.5 rounded-xl bg-indigo
      -600 px-4 py-2.5 text-sm font-semibold text-white shadow-sm transition-all duration-150 hover:bg-ind
      igo-500 hover:shadow-[0_4px_16px_-6px_rgba(79,70,229,0.6)] active:scale-[0.98] disabled:cursor-not-a
      llowed disabled:opacity-50 dark:bg-indigo-500 dark:hover:bg-indigo-400;
15744   }
```

```
15745
15746   /* 次按钮 / 幽灵按钮 */
15747   .btn-ghost {
15748     @apply inline-flex cursor-pointer items-center justify-center gap-1.5 rounded-xl border bo
      rder-gray-200 bg-white px-4 py-2.5 text-sm font-medium text-gray-700 transition-colors hover:bg-gray
      -50 active:scale-[0.98] disabled:cursor-not-allowed disabled:opacity-50 dark:border-slate-700 dark:b
      g-slate-800 dark:text-slate-200 dark:hover:bg-slate-700;
15749   }
15750
15751   /* 危险按钮（浅底） */
15752   .btn-danger {
15753     @apply inline-flex cursor-pointer items-center justify-center gap-1.5 rounded-xl bg-red-50
       px-4 py-2.5 text-sm font-medium text-red-600 transition-colors hover:bg-red-100 active:scale-[0.98]
       disabled:opacity-50 dark:bg-red-500/10 dark:text-red-400 dark:hover:bg-red-500/20;
15754   }
15755
15756   /* 输入框 */
15757   .input {
15758     @apply w-full rounded-xl border border-gray-200 bg-white px-3.5 py-2.5 text-sm text-gray-9
      00 transition-colors placeholder:text-gray-400 focus:border-indigo-500 focus:outline-none focus:ring
      -4 focus:ring-indigo-500/15 dark:border-slate-700 dark:bg-slate-800 dark:text-slate-100 dark:placeho
      lder:text-slate-500 dark:focus:border-indigo-400 dark:focus:ring-indigo-400/15;
15759   }
15760
15761   /* 表单字段标签 */
15762   .label {
15763     @apply mb-1 block text-sm font-medium text-gray-700 dark:text-slate-300;
15764   }
15765
15766   /* 分组小标题（SectionLabel / GroupLabel 通用） */
15767   .section-label {
15768     @apply px-1 pt-4 text-[11px] font-medium uppercase tracking-wider text-slate-400 dark:text
      -slate-500;
15769   }
15770
15771   /* 分段选择器（登录 Tab 等） */
15772   .seg {
15773     @apply flex rounded-xl bg-gray-100 p-1 dark:bg-slate-800;
15774   }
15775   .seg-item {
15776     @apply flex-1 cursor-pointer rounded-lg py-2 text-sm font-medium transition-all;
15777   }
15778   .seg-item-active {
15779     @apply bg-white text-indigo-600 shadow-sm dark:bg-slate-700 dark:text-indigo-400;
15780   }
15781   .seg-item-idle {
15782     @apply text-gray-500 hover:text-gray-700 dark:text-slate-400 dark:hover:text-slate-200;
15783   }
15784
15785   /* 数值大标题（统计数字） */
15786   .num {
15787     @apply font-bold tabular-nums tracking-tight;
15788   }
15789 }
15790
```